



**EXPLORING MODELS, DATA STRATEGIES, AND HYPERPARAMETER
TUNING IN A TRIP RECOMMENDATION SYSTEM**

Necati ERKAL

FEBRUARY 2024

ÇANKAYA UNIVERSITY

GRADUATE SCHOOL OF NATURAL AND APPLIED SCIENCES

DEPARTMENT OF COMPUTER ENGINEERING

M.Sc. Thesis in

INFORMATION TECHNOLOGIES



**EXPLORING MODELS, DATA STRATEGIES, AND HYPERPARAMETER
TUNING IN A TRIP RECOMMENDATION SYSTEM**

Necati ERKAL

FEBRUARY 2024

ABSTRACT

EXPLORING MODELS, DATA STRATEGIES, AND HYPERPARAMETER TUNING IN A TRIP RECOMMENDATION SYSTEM

ERKAL, NECATİ

M.Sc. in Information Technologies

Supervisor: Assist. Prof. Dr. A. Nurdan SARAN

February 2024, 72 pages

Recommender system's importance has been increasing recently. It is becoming more difficult to make a recommendation that users might like because of the complex data. Especially in trip recommender systems, recommending the next city is a challenging task, where recommending truly is essential. According to various studies, deep learning with recommender systems helps improve recommendations' accuracy and handle complex data. This study investigates the impact of data augmentation, model architecture modifications, and hyperparameter tuning on the performance of a deep learning-based recommendation system. The research begins by exploring data augmentation techniques, specifically increasing dataset diversity through trip adjustments. Contrary to expectations, these augmentations led to a progressive decrease in model performance. Demonstrating the subtle impact of architectural changes on model effectiveness, the study finds that removing particular layers negatively impacts performance while adding a specific layer slightly improves results. In addition, hyperparameter tuning identifies optimal configurations that slightly improve model scores while maintaining reasonable run times and loss levels. This research highlights the complexity of designing efficient and accurate recommendation systems, emphasizing the need for a balanced approach in data handling, model architecture, and parameter selection.

Keywords: Recommender Systems, Data Augmentation, Feature Engineering, Trip Recommendation, Deep-Learning Based Recommender Systems.



ÖZET

**BİR SEYAHAT ÖNERİ SİSTEMİNDE MODELLER, VERİ STRATEJİLERİ
VE HİPERPARAMETRE AYARINI KEŞFETMEK**

ERKAL, NECATİ
Bilgi Teknolojileri Yüksek Lisans

Danışman: Dr. Öğ. Üyesi A. Nurdan SARAN
Şubat 2024, 72 sayfa

Öneri sistemlerinin önemi son dönemde giderek artıyor. Verilerin karmaşıklığı nedeniyle kullanıcıların beğenebileceği bir öneride bulunmak giderek zorlaşıyor. Özellikle gezi tavsiyesi sistemlerinde, tavsiye etmenin gerçekten önemli olduğu bir sonraki şehri tavsiye etmek zor bir iştir. Çeşitli araştırmalara göre, derin öğrenmeyi öneri sistemleriyle birlikte kullanmak, önerilerin doğruluğunu artırmaya ve karmaşık verileri yönetmeye yardımcı oluyor. Bu çalışma, veri artırmanın, model mimarisi değişikliklerinin ve hiperparametre ayarının derin öğrenmeye dayalı bir öneri sisteminin performansı üzerindeki etkisini araştırıyor. Araştırma, veri artırma tekniklerini keşfederek, özellikle de gezi ayarlamaları yoluyla veri kümesi çeşitliliğini artırarak başlıyor. Beklentilerin aksine, bu artışlar model performansında giderek artan bir düşüşe yol açtı. Mimari değişikliklerin model verimliliği üzerindeki ince etkisini gösteren çalışma, belirli katmanların kaldırılmasının performansı olumsuz etkilediğini, belirli bir katmanın eklenmesinin ise sonuçları biraz iyileştirdiğini ortaya koyuyor. Ek olarak, hiperparametre ayarı, makul çalışma sürelerini ve kayıp seviyelerini korurken model puanlarını bir miktar artıran optimum konfigürasyonları tanımlar. Bu araştırma, veri işleme, model mimarisi ve parametre seçiminde dengeli bir yaklaşım ihtiyacını vurgulayarak verimli ve doğru öneri sistemleri tasarlamamanın karmaşıklığını vurgulamaktadır.

Anahtar Kelimeler: Öneri Sistemleri, Veri Arttırımı, Özellik Mühendisliği, Seyahat Önerisi, Derin Öğrenme Tabanlı Öneri Sistemleri.

ACKNOWLEDGEMENT

Special thanks to my supervisor, Assist. Prof. Dr. A. Nurdan SARAN for the excellent guidance and for providing me with an excellent atmosphere to conduct this research. I offer my gratitude for her patience and support.

I express my gratitude to my friends Alper Emre HAS and Aydın DOĞAN, who provided assistance and inspiration during this study.

I want to thank my wife, Ayça ERKAL, and my son, Efe ERKAL, for their support and encouragement.

TABLE OF CONTENTS

STATEMENT OF NONPLAGIARISM	iii
ABSTRACT	iv
ÖZET.....	vi
ACKNOWLEDGEMENT	vii
LIST OF TABLE	x
LIST OF FIGURES	xi
LIST OF ABBREVIATIONS	xiii
CHAPTER I.....	1
INTRODUCTION.....	1
1.1 OBJECTIVES AND CONTRIBUTIONS	1
1.2 ORGANIZATION OF THE STUDY	2
CHAPTER II	3
BACKGROUND	3
2.1 RECOMMENDER SYSTEMS.....	3
2.2 DEEP LEARNING IN RECOMMENDER SYSTEMS	4
2.2.1 User-Item Interactions in Deep Learning Models	5
2.3 ENSEMBLING AND MODEL COMBINATION	6
2.4 FEATURE ENGINEERING, PRE-PROCESSING, SELECTION AND GENERATION	7
2.5 SESSION-BASED MATRIX FACTORIZATION.....	7
2.6 MULTILAYER PERCEPTRON WITH SESSION-BASED MATRIX FACTORIZATION	8
CHAPTER III	9
LITERATURE REVIEW.....	9
CHAPTER IV.....	13
METHODOLOGY.....	13
4.1 DATASET.....	13
4.1.1 Pre-Processing and Feature Engineering	15
4.1.2 Data Augmentation	19
4.2 METHODS.....	22

4.2.1 Environment Requirements	22
4.2.2 Algorithm.....	23
CHAPTER V	32
EXPERIMENTAL RESULTS	32
5.1 FEATURE ENGINEERING AND DATA AUGMENTATION	32
5.2 HYPERPARAMETER TUNING AND MODIFICATIONS TO MODELS	
33	
5.2.1 Initial Model.....	34
5.2.2 1 Layer Removed Model.....	35
5.2.3 1 Layer Added Model	38
5.2.4 2 Layers Added Model.....	39
5.2.5 RNN Added Model	40
5.2.6 GRU Added Model	41
5.2.7 Hyper Parameter Tuning Experiments.....	43
CHAPTER VI.....	46
DISCUSSION AND CONCLUSION	46
REFERENCES.....	52

LIST OF TABLE

Table 1: Dataset Description	14
Table 2: Feature Categories	17
Table 3: Feature Engineering of Our Study.....	18
Table 4: Yaml File Parameters	31
Table 5: Data Experiment Results.....	32
Table 6: Balance of Datasets	33
Table 7: Model Experiment Results.	33

LIST OF FIGURES

Figure 1: test_set.csv Data Sample.....	15
Figure 2: train_set.csv Data Sample.....	16
Figure 3: train_and_test2.csv Data Sample.....	16
Figure 4: Trip Data Sample.....	20
Figure 5: Reversed Trip Data Sample.....	20
Figure 6: Our Data Augmentation Sample.....	21
Figure 7: Initial MLP-SMF Model.....	24
Figure 8: 1 FC Layer Removed MLP-SMF Model	26
Figure 9: 1 FC Layer Added MLP-SMF Model	27
Figure 10: 2 FC Layers Added MLP-SMF Model	28
Figure 11: GRU Added 1 FC Layer Removed MLP-SMF Model	29
Figure 12: RNN Added 1 FC Layer Removed MLP-SMF Model	29
Figure 13: Dropout Regularization	30
Figure 14: Initial Model Train and Validation Loss Comparison	34
Figure 15: Initial Model Score	35
Figure 16: 1 FC Layer (Layer 2) Removed Model Train and Validation Loss Comparison	36
Figure 17: 1 FC Layer (Layer 2) Removed Model Score	36
Figure 18: 1 FC Layer (Layer 3) Removed Model Train and Validation Loss Comparison	37
Figure 19: 1 FC Layer (Layer 3) Removed Model Score	37
Figure 20: 1 FC Layer Added Model Train and Validation Loss Comparison	38
Figure 21: 1 FC Layer Added Model Score	38
Figure 22: 2 FC Layers Added Model Train and Validation Loss Comparison	39
Figure 23: 2 FC Layers Added Model Score	40
Figure 24: RNN Added Model Train and Validation Loss Comparison	40
Figure 25: RNN Added Model Score	41
Figure 26: GRU Added Model Train and Validation Loss Comparison	42

Figure 27: GRU Added Model Score	42
Figure 28: Best Scores in Hyperparameter Tuning.....	43
Figure 29: Overall Experiments in Hyper Parameter Tuning Score vs. Index of Runs.....	44
Figure 30: Overall Parameters and Values in Hyper Parameter Tuning	44



LIST OF ABBREVIATIONS

ABBREVIATIONS

RecSys	: Recommender Systems
GRU	: Gated Recurrent Unit
RNN	: Recurrent Neural Network
GPS	: Global Positioning System
BayDNN	: Bayesian Personalized Ranking Deep Neural Network
SDAE	: Stacked Denoising Auto Encoders
QPR	: Quadratic Polynomial Regression
DNN	: Deep Neural Network
CF	: Collaborative Filtering
CNN	: Convolutional Neural Network
DRNN	: Deep Recurrent Neural Network
DAN	: Deep Attention Neural Network
ANN	: Attention Based Neural Network
SMF	: Session Based Matrix Factorization
MLP	: Multi-Layer Perceptron
WSDM	: Web Services Distributed Management
CSV	: Comma Separated Values
SQL	: Structured Query Language
RAM	: Random Access Memory

CHAPTER I

INTRODUCTION

Recommender systems, which are known as RecSys, play an essential role in shopping websites, social media platforms, news apps, and online video streaming sites. NVIDIA has introduced a tool called Merlin to help deep learning recommender systems. NVIDIA's teams have won two RecSys competitions. The first was the ACM RecSys Challenge in 2020; the second was the WSDM WebTour 21 Challenge organized by Booking.com. This challenge was about predicting a traveler's last destination in their trip based on their former booking reservations. NVIDIA's team included Kaggle Grandmasters, Data Science experts from RAPIDS, and the Merlin Team [1].

This study focuses on this recommendation system that performed on a complex dataset presented by Booking.com. Various data augmentation, feature engineering, architecture, and hyperparameter tuning experiments have been made to enhance the study. Each row represents a city trip on data. The trips have been grouped by trip ID numbers, which are unique for each of them.

1.1 OBJECTIVES AND CONTRIBUTIONS

Our study aims to provide better-personalized travel recommendations by using complex models and algorithmic techniques to understand user behavior and preferences. The study focuses on modeling relationships between trips and user preferences. Contributions of the research include model optimization and hyperparameter tuning. Testing of various deep learning architectures such as multilayer perceptron and its variations, simple RNN, and GRU as model optimization; performing experiments using the Bayesian method and finding the optimal parameter combinations to maximize model performance have been conducted and get better results. Performance analysis of the results has been made to obtain critical findings that increase the accuracy and efficiency of the recommendation system by analyzing each model trial at the end of the study.

How can the integration of complex models, algorithmic techniques, and deep learning architectures, such as multilayer perceptron, simple RNN, and GRU, be optimized with hyperparameter tuning to enhance the score of trip recommendations? What are the effects of data augmentation on the overall generalization of a model? This thesis aims to answer the questions above.

1.2 ORGANIZATION OF THE STUDY

This study has been divided into five chapters. Chapter I gives concise information about our study. Chapter II explains recommender systems and related works that have been conducted before. Chapter III has a detailed explanation of how the study has been conducted and how each step has been done. Chapter IV has graphics and an explanation of the experimental results. Finally, Chapter V has detailed information about the discussion of each result and conclusion part.

CHAPTER II

BACKGROUND

2.1 RECOMMENDER SYSTEMS

Recommendation Systems collect information about users' likes, such as movies, songs, books, and travel destinations. They manage this data in two ways: by asking users to rate things or by watching what users do, like which songs they listen to or which websites they visit. These systems may consider user details like age, where they come from, and their gender. In the online world, they use information from social media, like who follows you, what you write, and what you like. Using data from smart devices, like GPS locations and real-time health information, is popular [2].

According to [3], in a recommender system, recommendations are offered, which the system gathers and gives to the correct receiver. In some cases, the issue is how the recommendations are combined. The value of a system is in its capacity to establish effective connections between those who make suggestions and those who need suggestions. Recommenders help users choose things they might like from a narrowed, comprehensive list. They try to predict what a user might want and make their choices more accessible and better. It's like taking song suggestions that match your likes or things you'd like to buy without searching for hours. Recommender systems make this possible. Entertainment platforms use them to create playlists, suggest movies, and entertain us with things we like. Social media helps us to find interesting content. In education, they can recommend courses that match our likes.

Recommender systems categorized into three types. The first one is Content-Based Recommendation methods. These methods use user profiles and product descriptions to recommend items by searching former user behaviors and preferences. Secondly, Collaborative Filtering-Based Recommendation methods work by looking at past user behaviors without using personal information or detailed product descriptions. Recommendations are produced based on user ratings and behaviors. To handle the difficulties of these two methods, Hybrid Recommendation

Methods emerged. By combining both content-based and collaborative filtering-based methods, these hybrid methods aim to increase accuracy [11].

In [4], authors compared traditional methods and hybrid methods; Collaborative Filtering (CF), Demographic-Based Filtering, Knowledge-Based Mechanisms, and Content-Based Filtering Mechanisms are the main methods that researchers use for recommender systems. Collaborative Filtering (CF) is a prevalent method in Recommender Systems. The Demographic-Based Filtering algorithm utilizes the demographic information of users, including their location, language, age, religion, and other parameters. They use this data to match items with users and make the recommendations closer to the user's choices. The Knowledge-Based Filtering recommender approach relies on having clear standards for making suggestions. Finally, they define Content-Based Filtering as a method that recommends items or products that are similar to items the user previously liked or is interested in.

The authors in [5] and [7] found that at the beginning of recommendation system development, traditional methods played an important role. Two main approaches were defined as traditional methods: collaborative filtering and content-based filtering, which are the basis of new recommendation techniques.

2.2 DEEP LEARNING IN RECOMMENDER SYSTEMS

Deep learning in recommender systems contains neural network architectures. Their goal is to enhance the performance of recommendation algorithms. These models use neural networks with multiple layers to get features and relationships in input data. Deep learning-based recommendation systems can work with various types of information, like user preferences, items, and data. The use of embeddings helps to get representations of user-item interactions and makes the model generalize well to features that are not obvious. Deep learning has shown enhancements in recommendation quality in many types of research.

In [8], the authors point out that online information continues to grow, and recommender systems have appeared as a solution to deal with large amounts of data. These systems point to issues related to too much information and too many options. According to them, in recent times, deep learning has gotten attention in various research areas like computer vision and natural language processing. This is because deep learning not only works well but also can build hidden features. Deep learning has made an impact in getting information in recommender systems.

The authors researched in [9] that with less feature engineering, deep neural networks can perform better in increasing accuracy. This is achieved by creating embeddings for sparse features. They say neural networks using embeddings may make less relevant suggestions when interactions between users and items are too complicated. They introduce a method for training comprehensive linear models and deep neural networks, which aims to use memorization and generalization views to enhance recommender systems.

In [10], the authors compared many model architectures and discussed a study on Netflix's recommendation tasks. According to them, deep-learning models did not initially show significant improvements over non-deep-learning approaches. However, this changed when a detailed feature engineering was made on the input data. The study also indicates deep learning achieved considerable enhancements in their recommendations.

In [11], the author highlights the growing capacity of data from electronic and automated devices and emphasizes the need for intelligent systems to process and analyze this data for user benefit. Deep learning-based recommender systems are suggested as solutions to point to this challenge and make it easier for information gathering.

In [12], the authors discuss YouTube's recommender system. The author emphasizes that important performance increases are achieved by deep learning. It is designed around the two-staged information-gathering framework, with the candidate generation model in the first part and the deep ranking model in the second part.

2.2.1 User-Item Interactions in Deep Learning Models

Multi-layered neural networks get information and make understanding how things are connected easier. In recommender systems, this means deep learning models can understand the relationships between how users behave and what they like. Their ability to process all kinds of data makes deep learning models more powerful. They can work with structured data and also unstructured data. This allows them to find hidden patterns. Deep learning models are also good at understanding sequences. For example, they can follow how a user interacts with items over time. This is good when user behaviors depend on what they did before, like watching videos or making purchases. Deep learning models can remember past interactions and use that information to suggest better [13-16].

According to the author in [17], deep learning models understand complex patterns in user-item interactions by processing and analyzing big amounts of data to learn complicated structures. These models can understand non-linear and important relationships between users and items. They can find deep features and evaluate interactions more accurately. They can handle challenges like cold-start problems and make personalized suggestions. Neural networks that have multi-layers can discover user preferences.

2.3 ENSEMBLING AND MODEL COMBINATION

Ensemble methods in machine learning are approaches that increase model performance by combining the outputs of many base models. The main idea behind ensemble methods is based on aggregating the predictions of each model to make a more effective prediction [18].

Ensemble methods offer many advantages. They can increase prediction accuracy by reducing the effects of errors in each sub-model [19]. These methods combine different sub-models; each may be particularly good at specific tasks [20]. This combination allows them to make more dependable predictions. They increase model stability by reducing the choices in predictions where a single model can be overfitting or underfitting [21]. Ensemble methods can get complex relationships in the data that might be hard to catch for individual models. This capacity to get complex patterns increases the prediction accuracy [22-23].

The integration of ensemble techniques in machine learning includes several approaches for model combination. Two main methods are weighted averaging and stacking [18][24-25].

Weighted averaging includes setting different weights to the predictions of individual models and combining them according to these weights. The choice of weights can be based on the performance of each model on a validation dataset. The models with superior performance receive higher weights. Weighted averaging can be used when you know which models are better in different situations or parts of the data [18].

Stacking includes training a new model that learns to combine the predictions of base models. The predictions of base models work as input features for new models. The new model is trained on a different dataset. It learns a way to join the base model

predictions. Stacking can be used when the relationships between model outputs are complicated and non-linear [25].

Using ensemble techniques in recommender systems is useful for improving robustness and generalization. Recommender systems do not work well when they confront challenges such as sparse data, cold start problems, and developing user preferences. Ensemble methods help to solve these issues by combining different sub-models. Ensemble techniques can improve reliability in recommender systems by reducing the impacts of extraordinary or wrong predictions. They get more reliable recommendations. They can overcome the "cold start" problem by combining multiple models, including recommendations for users with limited historical data [26-28].

2.4 FEATURE ENGINEERING, PRE-PROCESSING, SELECTION AND GENERATION

Feature engineering is about selecting or creating meaningful inputs, known as features for machine learning models. In feature engineering, the goal is to improve model performance by simplifying data, decreasing the noise of information, and increasing predictions. The selection of input variables by ourselves based on field expertise is called conventional feature engineering. It is commonly used in traditional modeling. They may not be efficient for generalization. Extracting the features from data by using statistical methods or advanced techniques like deep learning is called data-driven feature engineering [29-30]. Pre-processing means the first steps of techniques that are applied to data. Pre-processing methods aim to increase the quality and reliability of the data by removing or decreasing the effects of various factors [31-32]. Selection is the way of choosing sub-features or variables from a dataset. Feature selection aims to define a task's most appropriate and informative features, such as classification while removing less important or unnecessary ones. Generation is about creating new features or variables. New features can be generated from the existing ones to provide information or get specific patterns in data [33-36].

2.5 SESSION-BASED MATRIX FACTORIZATION

Matrix factorization is a main machine learning technique used in recommendation systems. It divides a large user-item interaction matrix into lower dimensional matrices to discover hidden factors. These learned factors are used to provide personalized suggestions based on historical user-item interactions. Session-

Based Matrix Factorization is a recommendation system model focusing on guessing the next item in a user's session or visit. It uses traditional Matrix Factorization techniques to understand user behavior in sessions and make personal suggestions based on their choices [37-39].

2.6 MULTILAYER PERCEPTRON WITH SESSION-BASED MATRIX FACTORIZATION

A Multilayer Perceptron (MLP) is an artificial neural network that consists of multiple layers of fully connected neurons. Each layer contains one or more neurons. MLP is a feedforward multi-layer neural network, which means the data goes in one direction from the input layer through the hidden layers to the output layer. This architecture is used in many machine learning tasks like classification and regression. Matrix factorization (MF) was mentioned in the previous section. When MLP is combined with Session-Based Matrix Factorization (SMF), it means combining two approaches in a recommendation system. SMF helps in learning hidden features. These hidden features represent users and items according to their interactions in sessions. These features can get the logic of user preferences and item characteristics. The hidden features learned by SMF can be given as input to the MLP. Users' sessions can be thought of as vectors of these hidden features. They provide information about the users' historical interactions. MLP can work with these session representations. The MLP can get complicated patterns and relationships in the session data. The output of the MLP can make recommendations for the following items in a session. The MLP can predict user preferences by considering the individual session's features and by realizing patterns in the sequence of sessions and the users' historical behavior. When SMF is combined with an MLP, a hybrid recommendation system is created that uses the strengths of both methods. SMF helps to learn meaningful representations from historical interactions. MLP can capture complicated patterns and dependencies to make recommendations. This hybrid approach can be beneficial where you have precise interactions and sequential data. Using SMF and MLP together enhances the recommendation system's ability to provide suggestions to users [40-42].

CHAPTER III

LITERATURE REVIEW

Many research studies show that deep learning methods are effective, as examined in previous sections. In [43], the authors suggest a new recommender system that uses deep learning to predict. In [44], the authors offer a model named TrustDL, which combines trust information from social networks into the recommendation process to prevent common problems like cold start problems and data sparsity. This technique integrates dictionary learning with trust consistency regularization and trust embedding to enhance feature representation. It groups similar users. In [45], authors researched music recommender systems for extracting music item features from audio or metadata. The algorithm learns sequential music patterns from playlists and focuses on deep learning in music recommendation systems. In [46], the authors introduce a recommender system method that shows the limitations of traditional systems. They use an ensemble-based deep learning method that analyzes user ratings and comments on products. They make sentiment analysis for user ratings and comments on products. In [47], the study provides a systematic look at deep learning-based recommendations in e-learning. They studied course recommendations using recurrent neural networks, convolutional neural networks, and deep reinforcement learning. The study in [48] uses recurrent neural networks, convolutional neural networks as models, and a large dataset of Twitter dialogues. It focuses on suggesting suitable quotes. In [49], the authors introduce a model called Bayesian Personalized Ranking Deep Neural Network (BayDNN) for friend recommendation in social networks, which uses a convolutional neural network to extract features from network data. It contains a new personalized ranking approach to get user's preferences. The results show that BayDNN improves friend recommendation accuracy by % five on dataset Epinions. In [50], the authors aim to make better recommendations for less popular web services that don't get attention. They use a deep learning framework called stacked denoising autoencoders (SDAE) to analyze data from popular services. It tries to make better suggestions for less popular ones. The results show that their

The method does better than existing approaches. According to the study findings, this model also offers more accurate suggestions for the less known services.

The researchers introduced a new method [51] to improve recommendation systems. They combined a Quadratic Polynomial Regression (QPR) model, which considers relationships between features and overcomes missing data, in order to do that they use a Deep Neural Network (DNN) model. The QPR model gets hidden features of users and items, and the DNN model uses them to predict user ratings on items. They conducted experiments on three datasets that showed that the combined QPR-DNN model performed better than traditional recommendation algorithms in prediction accuracy. The study stated the effectiveness of their approach and discussed the effects of different parameters on its performance.

The research in [52] is about making news article recommendations. The authors want to suggest interesting news articles among recently published ones. Old articles that are not valid cannot be recommended according to task. The problem is recommending new articles and handling articles the system hasn't seen before. To achieve that, they use session-based RNN, which is like a memory that looks at what you have checked out recently, and a model (CNN) that can understand what you like. They tested this model on a real news dataset, and their model worked better than traditional models.

In [53], the authors suggest an online shopping recommendation system. They used a Deep Recurrent Neural Network (DRNN) to understand how people buy things on a website. They wanted to help users to find what they wanted. The DRNN looks at the sequence of web pages users visit before making a purchase. They created a tool for comparing the DRNN with different models to see the experimental results. They used a genetic algorithm. They tested their model on an online store called Koala. The results showed that the DRNN was better at giving accurate recommendations compared to other methods they tried.

The authors suggest a new recommendation model called DMFL, which combines deep learning, metric learning, and factorization machine techniques [54]. The model aims to improve recommendations by capturing linear and nonlinear relationships. The experiments are done on three datasets, which are MovieLens-20M, MovieLens-1M, and BookCrossing. Experimental results show that DMFM performs better than other models in accuracy in all the datasets.

The authors in [55] focus on improving news recommendation systems by a Deep Learning Meta-Architecture called CHAMELEON. The goal is to make users' experiences personal. The CHAMELEON architecture consists of two modules. The first one is for learning representations of news articles based on their text. The second one is for getting session-based recommendations using Recurrent Neural Networks (RNN). They want to predict the following article users may like to read in a session. Experiments between many session-based recommendation methods showed that the CHAMELEON method gave the best recommendations.

In [56], the study introduces a makeup recommendation system using a deep neural network approach guided by examples and rules. The framework consists of three stages. First, facial features related to makeup were classified using a structured coding approach. Then, these features are given into a deep neural recommendation model. Lastly, the automatic makeup analysis system that visualizes the recommended style shows the consequences. The system also provided the best results in facial trait recommendation classification and recommendation accuracy.

The study in [57] concerns data sparsity in traditional collaborative filtering techniques used in recommender systems. The problem is the user's attitude on rating a few existing items. This makes it difficult to predict their preferences. The proposed solution uses a feed-forward deep neural network called DeepRec with item embedding and a weighted loss function. The item embedding creates numerical vectors, and the weighted loss function aims to arrange the popularity of recommended items experiments on the App and Last. FM datasets gave better results according to classical algorithms.

The authors in [58] introduce a new approach called Deep Attention Neural Network (DAN). To express the challenge of personalized news recommendation, DAN uses a Convolutional Neural Network (CNN), Recurrent Neural Network (RNN), and attention mechanisms to enhance recommendation performance. The model uses an attention-based parallel CNN (PCNN) to combine user interest features with an attention-based RNN (ARNN) to get sequential features of user clicks and an attention-based neural network (ANN) to design user's interest. These components are combined to provide practical news recommendations.

The authors in [59] introduce a new deep multi-criteria collaborative filtering model for recommender systems. The model combines collaborative filtering with deep learning. The model has two stages: the first includes a deep neural network that

predicts criteria ratings based on user and item features. The second one uses another deep neural network to set the relationship between all ratings and criteria ratings.



CHAPTER IV

METHODOLOGY

In this chapter, details about our work have been presented. This study is about the MLP-SMF layer. This is the first algorithm of the winning hybrid algorithm of the NVIDIA Team, which has come first in booking.com’s WSDM WebTour 2021 Challenge and on the data they provide. In this study, all experiments have been done with saved checkpoints. How the data was obtained and enlarged, and the requirements for the algorithms designed are talked about in this chapter.

4.1 DATASET

The initial dataset includes features of accurate travel bookings made on Booking.com’s online platform. It consists of 1,545,502 reservations and 359,000 unique trips between 39,902 possible destinations. The dataset is anonymized and masked with IDs, which means it does not contain real names. The dataset has been given as train and test sets, which makes it easy to use. This dataset is different because it uses only completed trips that belong to real users [60].

The dataset initially includes the columns shown in Table 1. These columns help provide information for understanding user behavior and travel patterns. ‘user_id’ is a unique number given to each user. Instead of sharing their names, user IDs identify the bookers. ‘checkin’ and ‘check out’ columns show us the arrival and leave dates of a hotel reservation. ‘affiliate_id’ is an anonymized ID that tells us how users learn and book a hotel. ‘device_class’ is about whether a reservation is made from a desktop or mobile device. ‘booker_country’ and ‘hotel_country’ give masked information about countries where the user comes from and the hotel located in. This means the data uses fake names to hide the real country names. ‘city_id’ is a unique identifier for the hotel’s city, which is our main aim to predict. ‘utrip_id’ defines a group of bookings in the same trip that belongs to one user. The cities visited in a trip that belongs to one user can be seen in one unique ‘utrip_id.’ This data was the initial retrieved data.

Table 1: Dataset Description

COLUMN	DESCRIPTION
user_id	A unique number is given to each user.
check-in	The date of reservation starts.
checkout	The date of reservation ends.
affiliate_id	Anonymized ID represents the ways users learn and connect with a hotel.
device_class	Explanation of reservation was made from a desktop or a mobile device.
booker_country	Masked data for the user's country.
hotel_country	Masked data for the hotel country.
city_id	Unique identifier for the hotel's city.
utrip_id	Unique identifier for a group of multi-destination bookings within the same trip.

The dataset contains travel sequences and user booking patterns. It gives us information about users, their travel history, and the cities they booked in. The task is to make predictions that need to be completed quickly.

The dataset provides appropriate data for use in sequential recommendation and retrieval problems. The data has been masked with IDs to protect user privacy and business information. This approach follows the privacy standards while offering a sample of real travel information. Each data row represents a stayed reservation, grouped into unique trips identified through the 'utrip_id' column. The main question addressed in the dataset is guessing the last city by predicting the city_id, which is

hidden. Building a model that represents the cities that users visit and predicts the last city users visit based on this model is the primary purpose of this work.

The dataset has been used in the Booking.com data challenge in the WSDM WebTour Challenge. Many research teams joined and offered their solutions to the sequential next-city guessing problem.

4.1.1 Pre-Processing and Feature Engineering

Preprocessing is an essential phase in data analysis and machine learning, which sets data for our recommendation task. These procedures confront many problems like noise, missing values, and mismatches. Widely used preprocessing steps include data cleaning, normalization, handling missing values, and feature scaling. The goal is to improve the data quality and enhance the learning process's performance in preprocessing.

The preprocessing step starts with the code which imports libraries including 'pandas', and 'cudf' which is a GPU using the DataFrame library and modules from sci-kit-learn.

user_id	checkin	checkout	city_id	device_class	affiliate_id	booker_country	hotel_country	utrip_id
1000027	13.08.2016	14.08.2016	8183	desktop	7168	Elbonia	Gondal	1000027_1
1000027	14.08.2016	16.08.2016	15626	desktop	7168	Elbonia	Gondal	1000027_1
1000027	16.08.2016	18.08.2016	60902	desktop	7168	Elbonia	Gondal	1000027_1
1000027	18.08.2016	21.08.2016	30628	desktop	253	Elbonia	Gondal	1000027_1

Figure 1: test_set.csv Data Sample

user_id	checkin	checkout	device_class	affiliate_id	booker_country	utrip_id	city_id	hotel_country
1000066	21.07.2016	23.07.2016	desktop	9924	Gondal	1000066_2	56430	Urkesh
1000066	23.07.2016	25.07.2016	desktop	9924	Gondal	1000066_2	41971	Urkesh
1000066	25.07.2016	28.07.2016	desktop	9924	Gondal	1000066_2	5797	Urkesh
10000270	8.02.2016	9.02.2016	mobile	9452	The Devilfire Empire	10000270_1	50075	The Devilfire Empire
10000270	9.02.2016	10.02.2016	desktop	116	The Devilfire Empire	10000270_1	29207	Cobra Island
10000270	10.02.2016	19.02.2016	desktop	9924	The Devilfire Empire	10000270_1	44768	Cobra Island

Figure 2: train_set.csv Data Sample

This step starts with reading the data, which are given in two CSV files in Figure 1 and Figure 2. train_set.csv and test_set.csv have been put into GPU DataFrames, which are two-dimensional labeled data structures, by using cudf. Figures 1 and 2 show us reservation information like city, check-in, checkout date, user identity, trip ID, and a unique number for each trip.

user_id	checkin	checkout	city_id	device_class	affiliate_id	booker_country	hotel_country	utrip_id	istest	fold	submission	N	utrip_id	dcount	icount
1000027	13.08.2016	14.08.2016	8183	desktop	7168	Elbonia	Gondal	1000027_1	0	0	0	4	0	0	3
1000027	14.08.2016	16.08.2016	15626	desktop	7168	Elbonia	Gondal	1000027_1	0	0	0	4	0	1	2
1000027	16.08.2016	18.08.2016	60902	desktop	7168	Elbonia	Gondal	1000027_1	0	0	0	4	0	2	1
1000027	18.08.2016	21.08.2016	30628	desktop	253	Elbonia	Gondal	1000027_1	0	0	0	4	0	3	0
1000033	9.04.2016	11.04.2016	38677	mobile	359	Gondal	Cobra Island	1000033_1	1	0	0	5	1	0	4
1000033	11.04.2016	12.04.2016	52089	desktop	384	Gondal	Cobra Island	1000033_1	1	0	0	5	1	1	3

Figure 3: train_and_test_2.csv Data Sample

The 'istest' column is added to train and test datasets to split them into training and test parts in one CSV file. The datasets are joined and located in a data frame called 'raw.' The joined DataFrame is sorted according to the 'user_id' and 'checkin' columns in ascending order. A new column called 'fold' is added to 'raw' because k-fold cross-validation is designed to five folds according to 'utrip_id' groups. Each

row's fold information can be seen in the 'fold' column. The 'submission' column is added. In the test set, rows with city_ids have zero values marked by assigning one to its value in the 'submission' column. If a row is marked zero in the city_id value, it means that this city_id should be predicted. The column 'N' has been created in the preprocessing step. The column 'N' counts the number of places that have been visited in each trip. 'utrip_id_' column is made for factorizing the 'utrip_id' for later calculations. 'dcount' and 'icount' columns have been added for counting each trip's city visits. These columns are numbered starting from zero in ascending and descending orders. The last frame ('raw') is exported to a CSV file named 'train_and_test_2.csv' at the end of this process. In Figure 3, you see the final dataset's first six rows, named train_and_test_2.csv. In this step, travel data was processed, and some additional features, like the number of visits, were created by the team. The dataset is prepared for the training and test stages.

Table 2: Feature Categories

Definition	Features
categorical features	city_id, hotel_country, booker_country, device_class, affiliate_id
date features	day_of_week, week_of_year, month, isweekend, season, stay_length, days_since_last_booking (check-in previous check-in)
lag features	first_city_in_trip, lagged_cities, lag_countries (previous 5)

When the NVIDIA Team built their models, they added all the columns from the dataset. They do feature engineering in the code before the training phase, too. In Table 2, some new columns and the categories they belong to can be observed. For example, by using information from the check-in and check-out date columns, new features are made, like the day of the week when the booking occurred, week of year, month, and a flag that shows if it was a weekend date or not. Extra information like the length of stay and the time since the last booking was also produced and used.

Reservations order can be seen chronologically if the rows are sorted according to utrip_id and dcount in ascending order. It would help to create sequential features in feature engineering. These features have the information of visited cities in the

sequence, which considers the previous five cities booked and combines this information into the models.

Trip statistics are made to increase the model's understanding of each reservation sequence. These trip statistics include the number of city visits in a trip, the stay time of the entire trip, which is measured in days, and the order of each reservation in the sequence. This part of feature engineering has been taken from the team's existing study.

The `train_and_test_2.csv` file has been imported to the SQL Server Management Studio. After taking this data to SSMS, we worked on this data to understand the relationships between the 'city_id' columns and create new features according to this analysis. The feature engineering and data augmentation parts have been conducted in SQL.

'CurrentCity,' 'NextCity,' 'PairCount,' and 'Probability' columns have been created by a query. The 'PairCount' column is about the count of the current city and the next city matches in the data. The 'Probability' column shows the probability rate of the next city. The 'Probability' column's values have been calculated by dividing the row's 'PairCount' value by all pair counts.

The final result has been ordered by 'PairCount' in descending order, probability in descending order, and current and next city in ascending order. Feature engineering provides us with new features that we hope will help us understand the sequential patterns in the dataset. The final features are shown in Table 3.

Table 3: Feature Engineering of Our Study

COLUMN	DESCRIPTION
user_id	A unique number is given to each user.
Checkin	The date of reservation starts.
checkout	The date of reservation ends.

Table 3 Continued

affiliate_id	Anonymized ID represents the ways users learn and connect with a hotel.
device_class	Explanation of reservation was made from a desktop or a mobile device.
booker_country	Masked data for the user's country.
hotel_country	Masked data for the hotel country.
city_id	Unique identifier for the hotel's city.
utrip_id	Unique identifier for a group of multi-destination bookings within the same trip.
current_city	This column has the same values as the city_id column for showing the next and current cities together.
next_city	Identifies the next cities for each current city.
pair_count	Count of occurrences for each pair of consecutive city_id's. Indicates how frequently a specific transition between two cities occurs in the dataset.
probability	The ratio of the count of each specific city pair to the total count of pairs providing

Three embedding layers, which are NextCity_embedding, PairCount_embedding, and Possibility_embedding, have been added to the model by using these new features. These experiments have not been evaluated because of the poor results.

4.1.2 Data Augmentation

Data augmentation is a technique that used in machine learning. It aims to increase the range of a dataset by modifying the data.

The team used data augmentation to increase the dataset by doubling it with reversed trips. For example, the original dataset has a trip from Milan-Venice-Rome-

Naples, as shown in Figure 4. If the Naples-Rome-Venice-Milan trip has been reversed, as shown in Figure 5, it will be a new trip because it will have a different last node to guess. The order of city visits can be according to logical orderings, which are more common in the real world. People plan trips in a sequential and logical order. The plans are made according to distances and transportation. When the trips are reversed, this logical order is still protected.

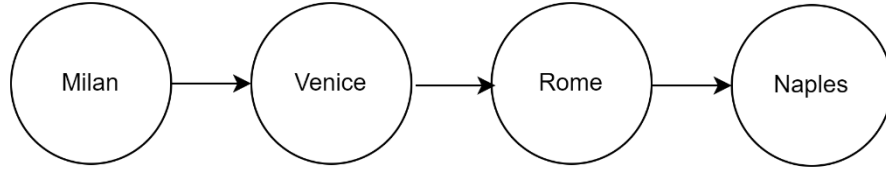


Figure 4: Trip Data Sample

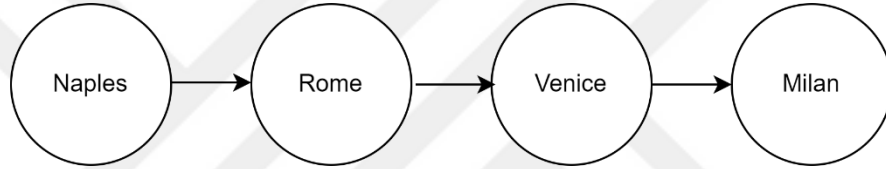


Figure 5: Reversed Trip Data Sample

According to them, they enhanced the accuracy score by doubling the dataset with reversed trips. Based on this idea, our data augmentation has been implemented to increase the data. This step of our study aimed to increase the dataset by adding new rows. Three iterations have been made to get three incremented datasets.

In the first iteration, the original dataset's number of rows has been increased by removing the first and last city visits from each trip, which has six or more visits. They have been added as new trips with new unique utrip_id's. Figure 6 shows a newly generated unique trip, Milan-Venice-Florence-Rome-Naples-Palermo, at the first step. In the second step, you see the first city, Milan, and the last city, Palermo, removed from the trip. You see a new trip in the third step, which is Venice-Florence-Rome-Naples, with the same logical and transportation order related to real-world situations. Reversing this new trip also creates a new trip. At the end of this iteration, the dataset has been extended from 1545502 rows to 1907466 rows. The dataset has been exported from SQL Server Management Studio by exporting the data as train_and_test_2_Deneme.csv to use in the training step.

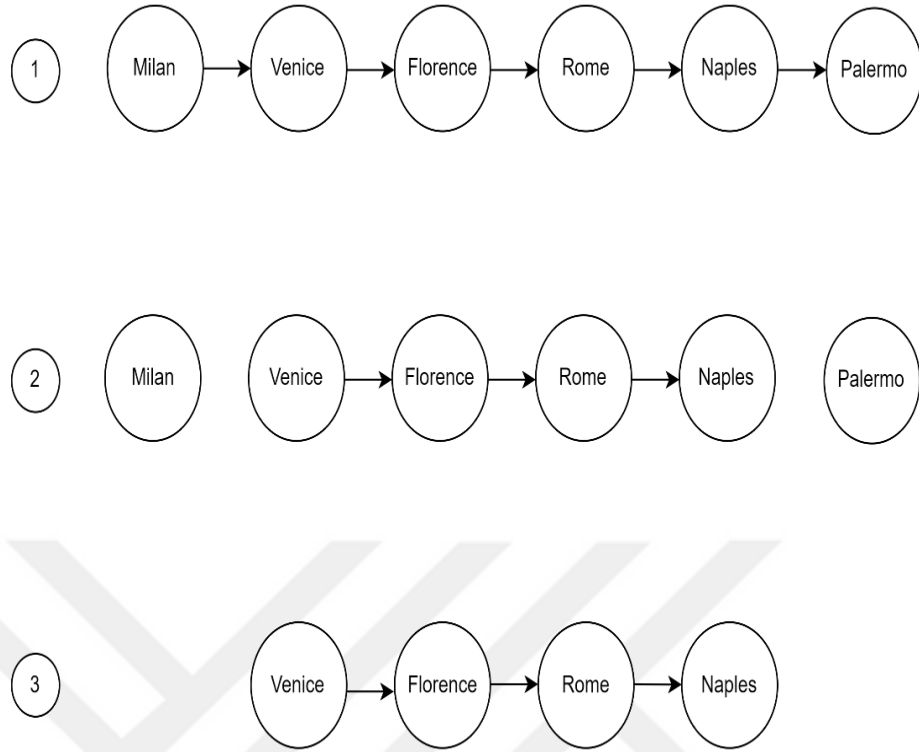


Figure 6: Our Data Augmentation Data Sample

In the first iteration, the trips only greater than six city visits have been taken and shortened them two nodes. The dataset still has trips greater than six city visits, which were greater than eight city visits before the removal of the first and last nodes. For the second iteration, the dataset has been extended to 2096681 rows and exported a new CSV file named `train_and_test_2_Deneme2.csv`. Then, in the third iteration, the new dataset has been extended to 2172282 rows and named `train_and_test_2_Deneme3.csv`.

In each iteration, a temporary table named `#filtered_trips` has been created in SQL Server Management Studio. This temporary table has been used to get rows from the original table where starting and end cities are not included. These filtered rows have been inserted into the original table, and the temporary table has been dropped. Two temporary tables have been created to update the `dcount` and `icount` columns. When removing the first and last nodes, `dcount` and `icount` column values do not start from zero. By updating the chronological order of trips and the reverse, the suitable `icount` and `dcount` values have been gotten for each added trip. The original table has been updated with these new counts.

After adding new trips, utrip_id values have been updated to unique values. To create these unique utrip_id values, new trips have been inserted into a temporary table named #t1. The maximum utrip_id of the existing dataset has been selected and inserted into another temporary table named #t2. New utrip_id values have been gotten by adding numbers in front of utrip_id's current value. For example, if our existing utrip_id value is 782095_1, one has been added as a prefix, and the new utrip_id value becomes 1782095_1 for the first iteration of data augmentation. In each iteration of data augmentation, a unique prefix has been used for the iteration. These new utrip_id values, which are in the #t1 table, have been used to update the utrip_id column in the original table. A backup table has been created to keep the original data. Then, the augmented dataset has been inserted into the main table.

4.2 METHODS

4.2.1 Environment Requirements

Our study has been conducted on two different systems. A computer that has Ubuntu 22.04 Linux and Windows 10 operating systems, one terabyte of storage, NVIDIA RTX 3070 Ti GPU, and 32 gigabytes of RAM has been used. When our code has been run, some problems have been emerged with the RTX 3070 Ti 's memory which is 8 gigabytes, and switched it with the RTX 3060 which has 12 gigabytes of memory. After this modification, the code has been executed, and our experiments have been made.

The dataset has been enlarged after three iterations of data augmentation. However, the code has not worked again because of out-of-memory exceptions. Our dataset experiments have been done on a server that has an NVIDIA RTX 4090 GPU with 24 gigabytes of storage. After this change, the dataset experiments have been made. Our system has enough resources for working on existing data, but a more powerful server has been needed while trying enlarged datasets.

Visual Studio Code (VSCode) has been used to run the code and create new models on the Ubuntu side, and SQL Server Management Studio on the Windows side to work on data and create enlarged ones.

The Miniconda installer has been downloaded to create an environment with the required libraries. The necessary execution permissions have been given. Then, the Miniconda installation process started. After the download had been completed, Conda needed to initialize our shell.

A Conda environment has been created and named `rapids-pytorch-cu12`. This environment has dependencies that are important for the code. It includes Rapids AI, PyTorch, and CUDA version 12.0. Rapids AI is a toolkit for data science and machine learning. It makes it easier to execute the code with the power of GPU. In version 23.08, Rapids AI includes cuDF, which is a tool for big data, cuML which uses graphics cards for learning patterns in data, cuGraph a tool for graphs that helps analyze relationships between them, cuSpatial which uses the graphics card to solve problems like finding points on a map, cuPy for doing math operations quicker by the graphics card, BlazingSQL a database tool using the graphics card to operate in large datasets.

The Jupyter Notebook has been converted into a Python file by a script. The `nbconvert` command has been used to convert the file type and use it in VSCode and Terminal as a Python file.

All our runs and results have been kept in an online platform wandb. Wandb has helped us log our runs and visualize the parameters that have been given to it. All the score and train-validation loss graphics have been generated in wandb automatically when a run or experiment has been started because it is logged in the code. Hyperparameter tuning experiments also have been made with the help of wandb.

4.2.2 Algorithm

The first layer of the study, a Multi-Layer Perceptron with Session-Based Matrix Factorization (MLP-SMF), has been used as the initial model. The initial model design is given in Figure 7.

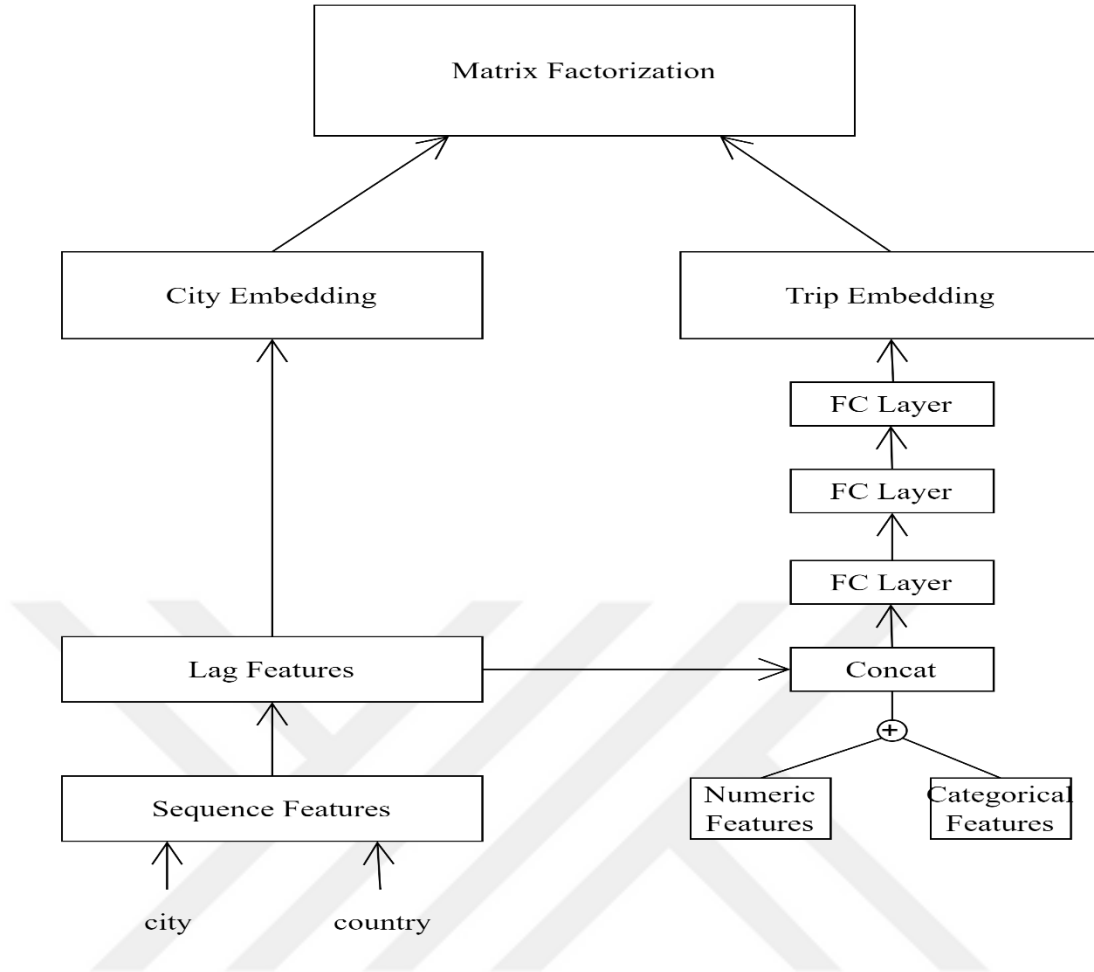


Figure 7: Initial MLP-SMF Model

The model takes information about our trips like the cities and hotels have been stayed at, arrival and leave dates and converts them into embeddings. Embedding is a technique used to assign numerical values to categorical variables or discrete elements. These numerical representations help the model understand the relationships between cities, hotels, and other representations. The model uses these embeddings and sets their sizes equally. Next, the model joins the embeddings from both types of features. Lag features, which keep our last five cities and last five countries, are also used for giving an embedding table directly to the matrix factorization. These lag features help the model remember the travel history of the last five cities and countries. Keeping the last five cities and countries helps to learn patterns in the order of visited cities. The model mixes all these embeddings, but it still keeps the sequence of cities ordered with the help of lag features. Mixed embeddings go through a series of layers. These steps help to create a summary of our recent travel session. With the help of this summarized session model, scores are given to different cities. Based on our travel history, these

scores help to suggest the next city to visit. MLP-SMF model converts our past travels to a summary and suggests new cities.

The model uses three linear layers, which have batch normalization, activation functions, and dropout, to get complicated patterns from the data.

5-fold cross-validation is used in the existing model. Cross-validation is a method that helps train a model. The data has been divided into five partitions, which are called folds. Each fold is used as the validation dataset one time. Rest folds are used for training. 80% of the data is used for training, while 20% is used for validation. This process is repeated five times. The score is counted by averaging all the folds' results.

Precision@4 is used for accuracy metrics. The model gives scores to cities; if the next city is in the top four cities with the highest scores, it is evaluated as true. If the consequences that come from Precision@4 are high, it means a successful recommendation. The model's ability to predict can be evaluated using this metric. If a user is about to plan a trip, the model suggests the next city; the model assigns a score to each city, and these scores are ranked. The Precision@4 metric evaluates whether the model can accurately predict the cities with the four highest scores. If it is considered that the model gives the cities rankings like Paris-Score: 0.9, Rome-Score: 0.8, Barcelona-Score: 0.7, Berlin-Score: 0.6 and Amsterdam-Score: 0.5, Precision@4 metric measures the model's ability to predict a ranking that includes the four highest scoring cities which are Paris, Rome, Barcelona, and Berlin. If the next city in the trip is among these four cities, the model is considered to have made a correct recommendation.

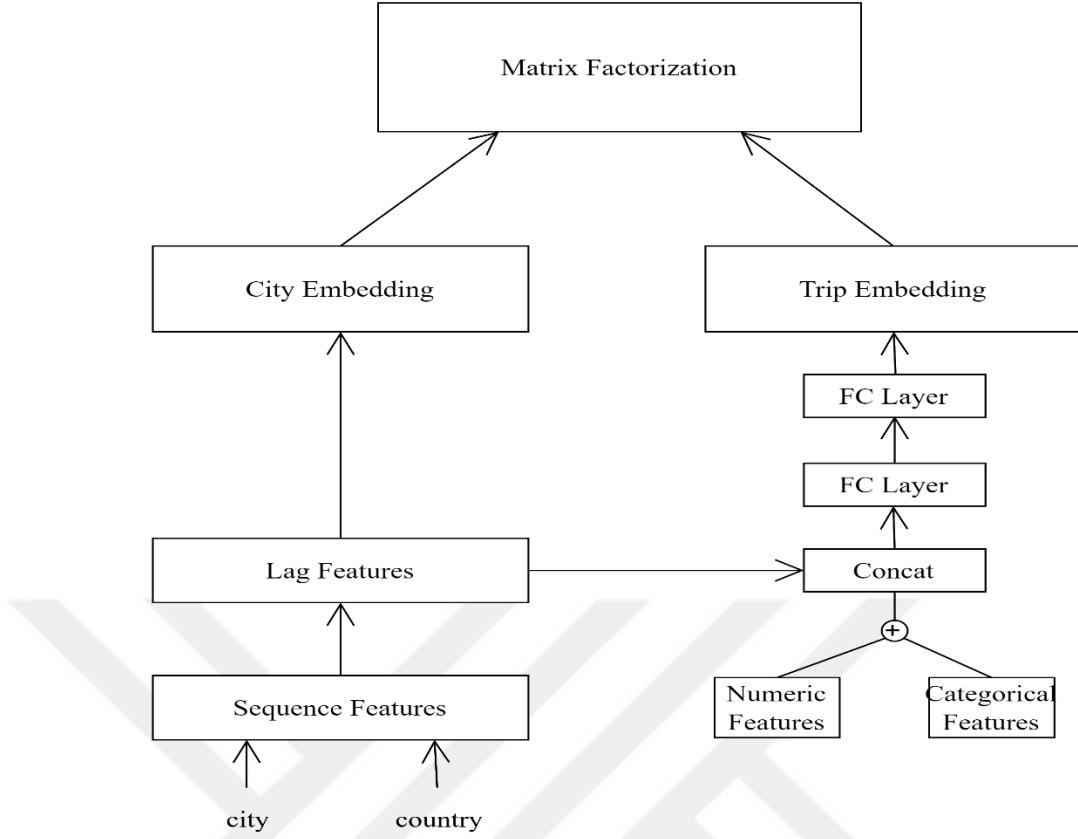


Figure 8: 1 FC Layer Removed MLP-SMF Model

In Figure 8, an FC layer has been removed to simplify the neural network architecture. The number of useful parameters increases model capacity in learning. Each layer causes more parameters in complex neural networks. However, higher bias may cause the model overfitting. This experiment has been conducted to see the results of how decreasing the number of fully connected layers affects accuracy. Our aim is to create a more generalized model in all model experiments. Less parameters or layers could lead to less computational load during the training. By the way, it is possible to evaluate the results if simplifying the architecture reduces the training time. In [61], the authors introduce a technique called “network trimming” for optimizing deep architectures. Network trimming is a study about the observation of neural networks that have a layer that shows no activation (zero activation). Zero activation layer means the layer that has no effect on the prediction success of the model. Removing these zero activation layers can cause a simplified model without getting affected in accuracy. In simplifying the model experiment, we aim to test whether the model has a zero-activation layer. In each step, an FC Layer has been removed.

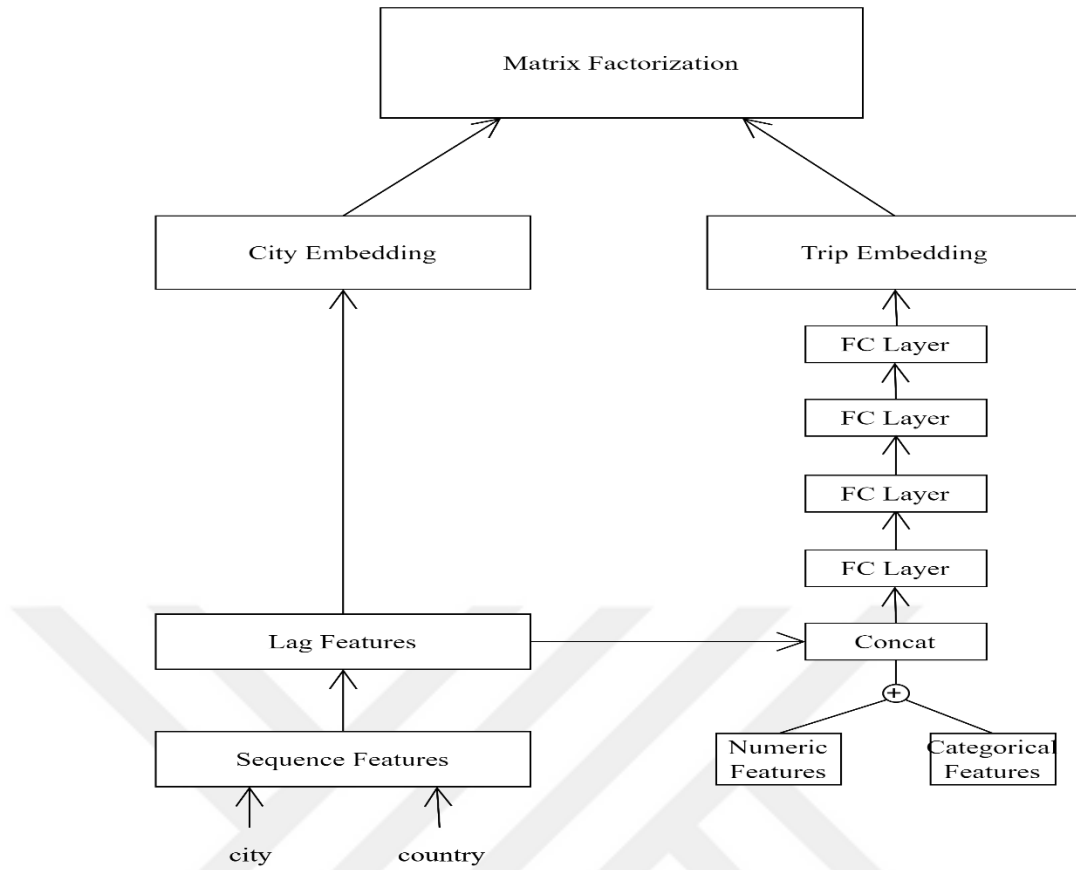


Figure 9: MLP-SMF Model with an added FC Layer

In Figures 9 and 10, layer-added models can be observed. In [62], authors search for optimal layer numbers in a neural network and the effects of adding layers to the model. They examine neural network layers for optimizing the accuracy and time complexity of complex problems. According to their study, they find that three-layer architecture has the best results in reducing time complexity and increasing overall score. They show that when the layers are less than three, the model's accuracy decreases, and when the layers are more than three, the model's time complexity increases in their study.

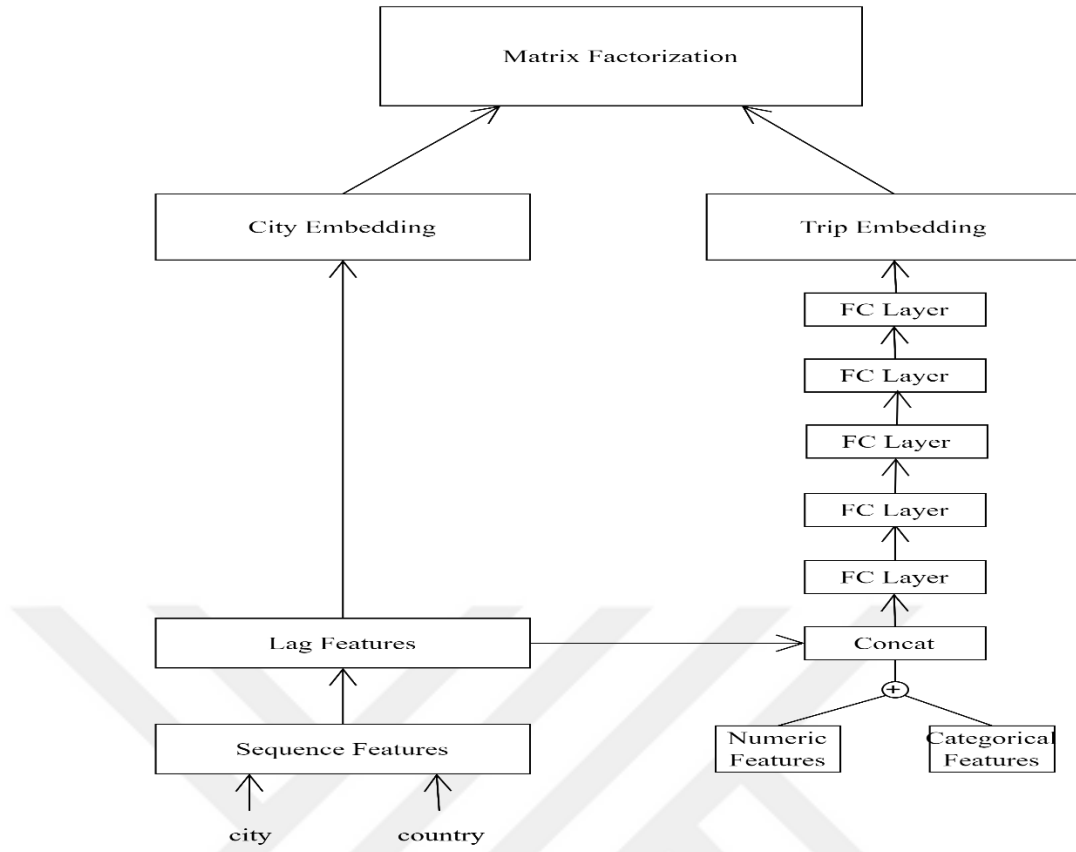


Figure 10: 2 FC Layers Added MLP-SMF Model

By adding layers to the model, our aim is to examine whether making our model deeper can achieve higher scores and less time complexity or training time. The model has been expected to learn better hierarchical features and complex representations by adding more fully connected layers.

In [63], the authors suggest an approach called the GRU-FC model; a GRU model with fully connected layers has been added inspired by this study. The aim is to see whether the results of adding GRU instead of a middle fully connected layer in the model increase the score. GRU model is a type of recurrent layer designed for sequential data. The data has sequential city trips, as mentioned before. In a trip recommender model where the order of cities is important, introducing a layer could enhance the model's ability to predict.

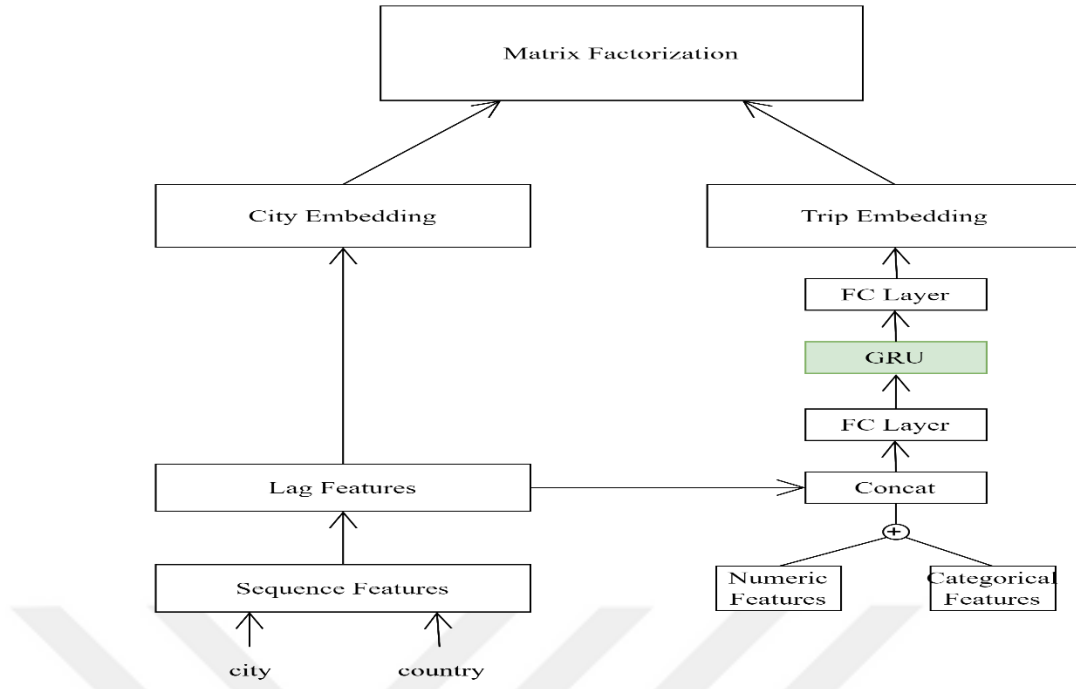


Figure 11: MLP-SMF Model that replaces an FC Layer with GRU

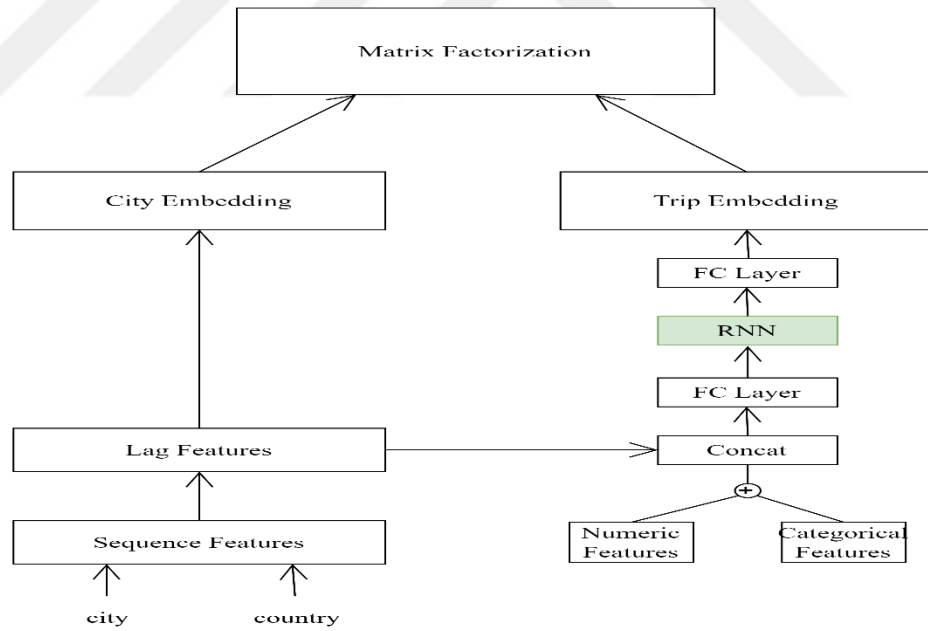


Figure 12: MLP-SMF Model that replaces an FC Layer with RNN

In Figure 11, a new model has been designed by removing the middle fully connected layer and relocating a recurrent neural network. In both the simple RNN-added model in Figure 12 and the GRU-added model instead of FC Layer experiments,

our aim is to see how different recurrent layers impact the model's ability to capture sequential patterns.

Regularization techniques have been applied in the team's study. Dropout is used as regularization.

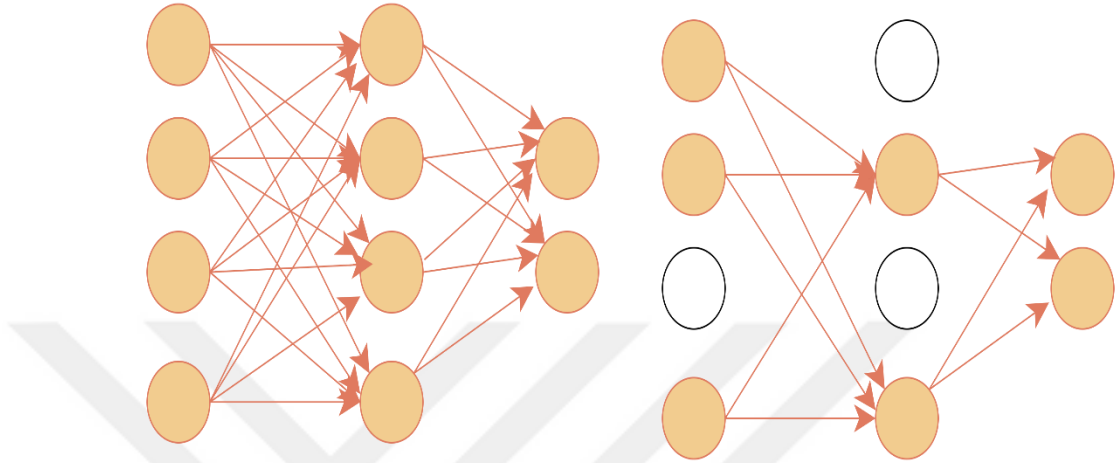


Figure 13: Dropout Regularization

Dropout is a regularization technique where random nodes are removed during training, which means they are temporarily deleted. In Figure 13, a fully connected model at the left and randomly removed nodes at the right are given. This helps to prevent overreliance on a node and provides a more generalized model. In the initial study, a dropout rate of 0.2 is used for the model. In order to try different dropout rates to find the balance that works best for our model and dataset, dropout rates 0.1, 0.2, 0.3, 0.4, and 0.5 have been applied.

Batch normalization is used after certain layers in the existing study. Every layer's input is fed with the normalized output vector of the layer, which comes before itself in batch normalization. It is expected to normalize the inputs of each layer and enhance the stability of the training process. Layer Normalization and Instance Normalization are implemented to test the effects of changing batch normalization.

Optimization techniques aim to improve the model's learning process. They help to get better performance. In the existing study, the Adam optimizer is used. For our sequential trip recommender, some other optimizers are used for sequential recommender tasks to analyze the effects of optimizers such as Root Mean Square Propagation (RMSprop), Stochastic Gradient Descent (SGD), Adaptive Gradient Algorithm (Adagrad), AdamW, and AdaDelta.

Some different parameter tests have been conducted on the scheduler, epochs, folds, batch size, and other parameters.

Table 4: Yaml File Parameters

Parameter	Values	Minimum Value	Maximum Value
Dropout Rate	-	0,05	1
Embedding Dimension	-	32	128
Epochs	-	6	24
Gradient Accumulation	-	1	2
Hidden Dimension	-	512	2048
Learning Rate	-	5E-05	0,002
Train Batch Size	-	128	512
Workers	-	4	16
Optimizers	Adam, SGD, AdamW, Nadam, RMSprop, Adagrad, Adadelata	-	-
Loss Function	CrossEntropyLoss, MseLoss, TripletMarginLoss, HingeEmbeddingLoss, BCEWithLogitLoss	-	-

Hyperparameter tuning experiments have been prepared using a YAML file to define the parameters such as dropout rate, embedding dimension, epochs, hidden dimension, learning rate, train batch size, workers, loss function, and optimizer, as in Table 4. Our goal is to maximize the overall score, expressed in the file with the appropriate format. The Bayes method is used in the YAML file while searching for the best combinations of parameters that give the highest score. The parameters range between minimum and maximum values. Choosing minimum and maximum values in this file based on best practices and the individual parameter experiments. For example, the epochs max value has been defined as ‘24’ because when it is tried in the individual parameter experiments, the score hasn’t been increased after 24 epochs.

CHAPTER V

EXPERIMENTAL RESULTS

In our study, with the purpose of achieving the best model that may produce the desired output, several experiments have been conducted to search for the optimal value of each parameter as different experiments. By the experiment setups described in the previous chapter that are related to individual parameter experiments, the best parameters have been listed as 0.2 for dropout value, Adam as an optimizer, Cross Entropy Loss as loss function, 12 for epochs, 0.001 as a learning rate, 5-fold, and batch size of 1024 which are already used in the proposed study. Different values of these parameters are searched and compared, and it has been decided to proceed with the proposed values for model and data experiments.

Architecture experiments also have been made. Fully connected layers have been added, removed, and relocated to simple RNN and GRU.

After finding the most generalized model with these values, hyperparameter tuning experiments have been conducted to find the best version of the parameter values that work together.

5.1 FEATURE ENGINEERING AND DATA AUGMENTATION

Data experiments have been conducted based on augmented data, which have been obtained in three versions in three iterations. In Table 5, the results of data augmentation and feature engineering experiments have been given. 1st iteration with the initial model offers the best performance.

Table 5: Data Experiment Results

Name	Precision@4 Score	Runtime
Data Augmentation 1st Iteration with Initial Model	0.554759	35 m 48 s
Data Augmentation 2nd Iteration with Initial Model	0.548075	36 m 11 s
Data Augmentation 3rd Iteration with Initial Model	0.545146	36 m 43 s

Table 6: Balance of Datasets

	Fold 0	Fold 1	Fold 2	Fold 3	Fold 4	Overall Row Number
Initial Data	313 649	309 164	305 575	307 546	307 568	1 545 502
1. Iteration Data	386 021	381 545	379 998	379 952	379 950	1 907 466
2. Iteration Data	423 893	419 386	417 851	417 789	417 762	2 096 681
3. Iteration Data	439 037	434 512	432 983	432 895	432 855	2 172 282

Table 6 shows the balance of all datasets obtained from data augmentation and used in the experiments. Each row shows a database, and each column indicates the number of records that the database has on each fold. The last column shows the overall sum of row counts.

5.2 HYPERPARAMETER TUNING AND MODIFICATIONS TO MODELS

As it is seen in Figure 7, the initial model's structure has three fully connected layers with session-based matrix factorization. Reduced layer experiments have been conducted to simplify the neural network architecture. In Table 7, the training loss (TR_loss), which is the difference between the model's predictions and actual labels in the train data, and validation loss (VL_loss) values of reducing the number of fully connected layers have been given.

Table 7: Model Experiment Results

Name	Precision@4 Score	Runtime	TR_loss	VL_loss
Initial Model Initial Data	0.566712	26 m 51 s	2.902	3.022
1 FC Layer Added Linear 21	0.567328	27 m 4 s	2.930	3.009
2 FC Layers Added Linear 21 Linear 22	0.565918	27 m 33s	2.966	3.009
1 FC Layer Removed 2nd	0.562569	37 m 4s	2.906	3.076
1 FC Layer Removed 3rd	0.540499	39 m 7 s	3.271	3.215
2nd FC Layer Removed GRU Added	0.561364	3 h 6 m 9 s	2.917	3.054
2nd FC Layer Removed RNN Added	0.541391	1 h 22 m 41 s	3.190	3.193

Layer addition experiments have also been made as one- and two-layer addition experiments. A fully connected layer named Linear21 has been located

between Linear2 and Linear3 layers. As seen in Table 7, the score increases slightly in this experiment. By adding one more layer to the model, the score falls.

One fully connected layer has been removed in each run. When the second fully connected layer is removed, the score decreases. The third fully connected layer-removing experiment shows a more dramatic fall in the score. Removing the middle fully connected layer and adding the simple RNN layer instead of this layer has a negative effect. Using GRU instead of a second fully connected layer also has a negative effect in terms of time.

5.2.1 Initial Model

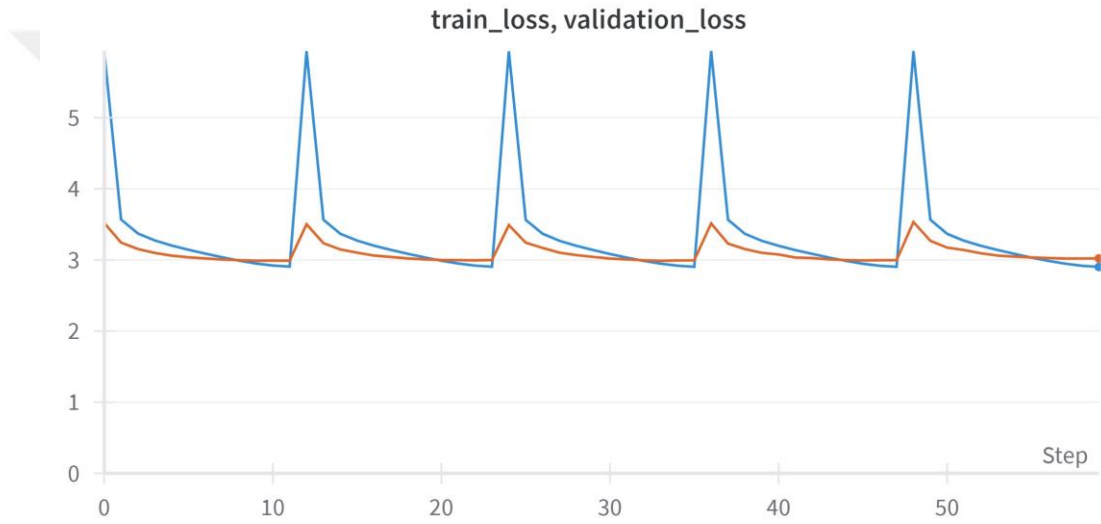


Figure 14: Initial Model Train and Validation Loss Comparison

In Figure 14, the initial model's train and validation loss graphics can be observed. The blue line represents train loss, while the orange line represents validation loss. The Y-axis shows us the values of train and validation loss. The X-axis shows us each step on the run, which means 5-fold 12 epochs for each experiment in this section.

Sharp and high train and validation loss values can be observed in each fold starting. Folds change in every 12 epochs, which means 11, 23, 35, and 47th steps. The train loss value can be observed as 2.902, and the validation loss value can be observed as 3.022 in this run.

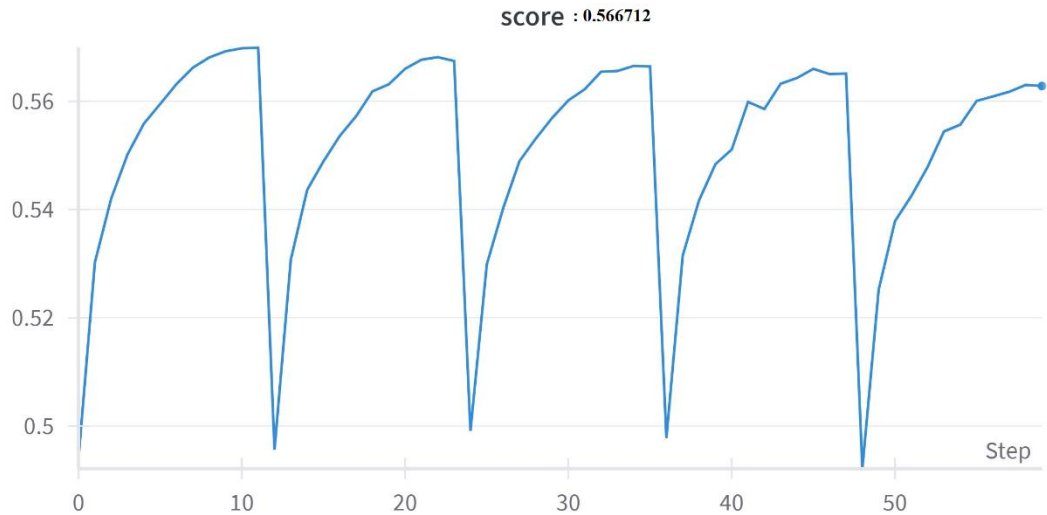


Figure 15: Initial Model Score

Figure 15 shows us the initial model's scores in every step. The Y-axis refers to score values, and the X-axis shows steps, which means the sum of epochs in every fold. This figure shows us score changes on epochs. The final score value is 0.566712, and the run time is 26 minutes 51 seconds.

5.2.2 1 Layer Removed Model

To simplify the neural network architecture, the number of fully connected layers has been removed to reduce the complexity of the model and get a more generalized model in this experiment. Fully connected layers add parameters to the model for enhancing the learning capacity. Layers 2 and 3 have been removed as different experiments to find a less effective layer.

In Figure 16, 1 FC Layer (Layer 2) removed the model's train, and validation loss values can be observed.

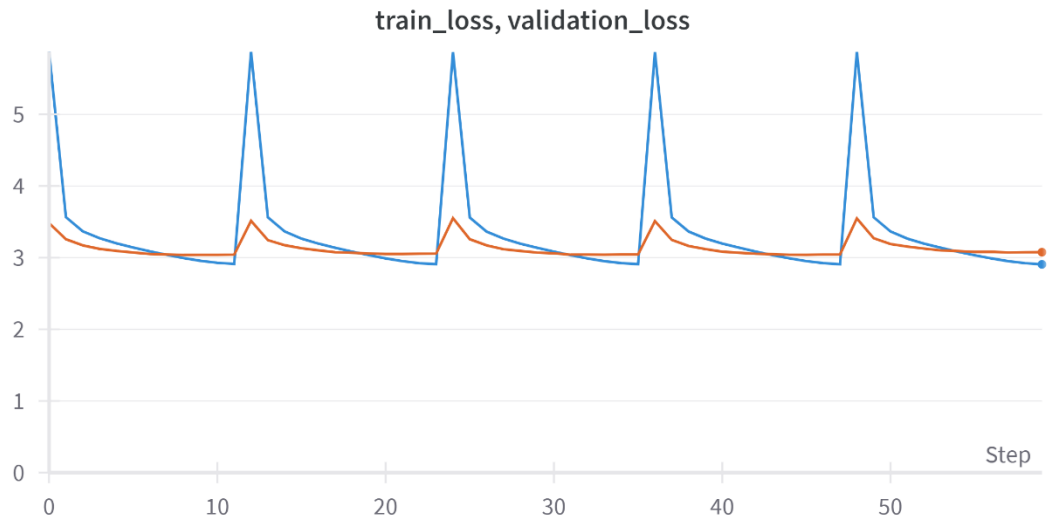


Figure 16: 1 FC Layer (Layer 2) Removed Model Train and Validation Loss Comparison

The train loss value can be observed as 2.906, and the validation loss value can be observed as 3.076 in this experiment.

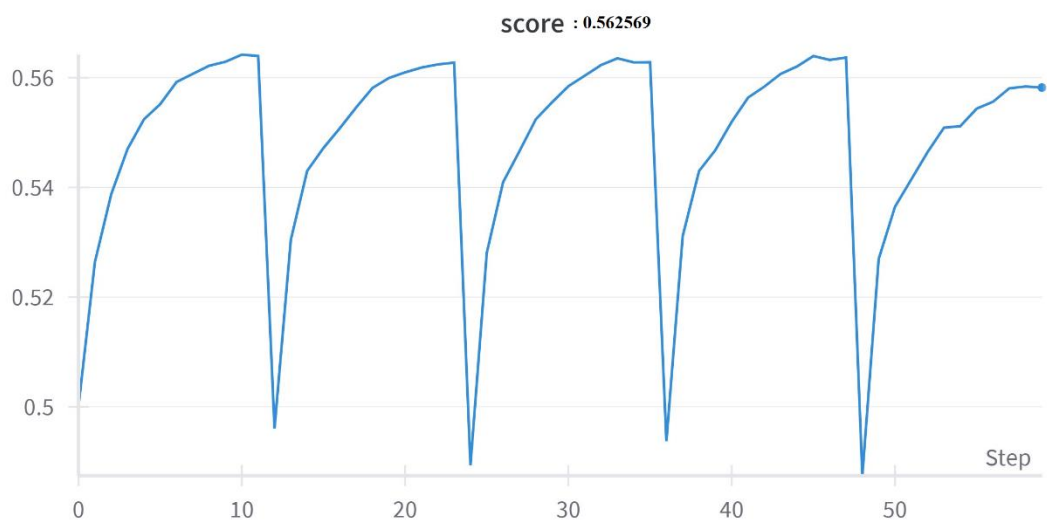


Figure 17: 1 FC Layer (Layer 2) Removed Model Score

Figure 17 shows us the score of the FC Layer 2 removed model experiment. The Y-axis shows score values in this experiment. The final score value is 0.562539, and the run time is 37 minutes 4 seconds.

Figure 18 shows the train and validation loss comparison chart of the Layer3 removed model. Train loss value of 3.271 and validation loss value of 3.215 have been obtained in this experiment.

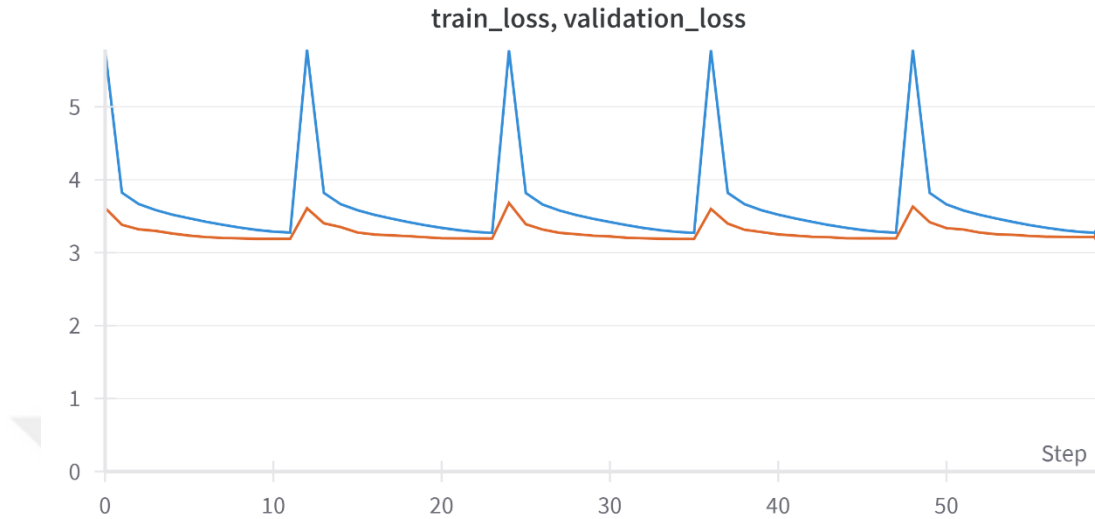


Figure 18: 1 FC Layer (Layer 3) Removed Model Train and Validation Loss Comparison

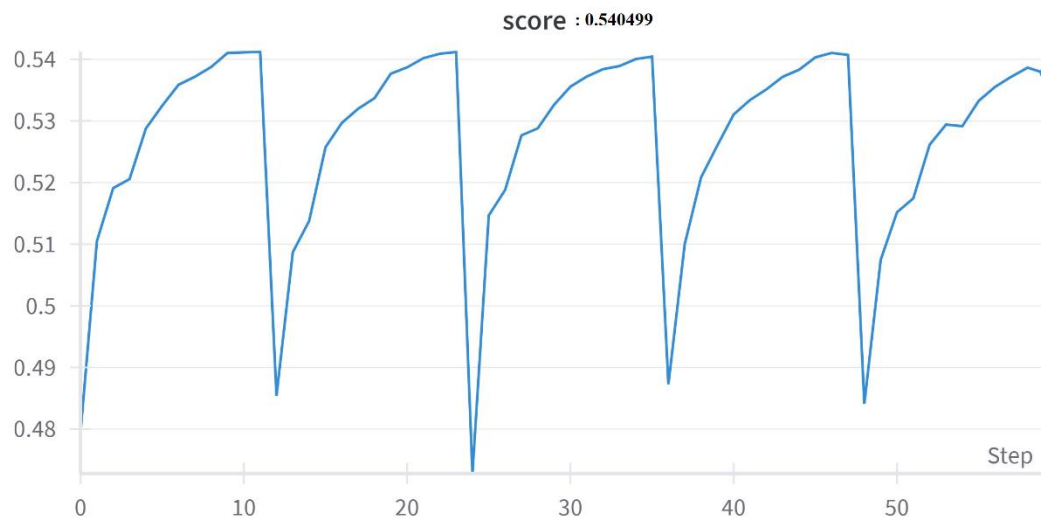


Figure 19: 1 FC Layer (Layer 3) Removed Model Score

Figure 19 shows the score of Layer3 removed model's score. In this experiment, the score falls to 0.540499 dramatically. The run time is 39 minutes and 7 seconds.

5.2.3 1 Layer Added Model

Fully connected layers help the machine learning models in enhancing learning capacity, generalization, and performance. Layer addition experiments have also been made to search for the optimal layer number. Finding the relationship between layer, score, and run-time values and the optimal value between them has been expected since the model learns better hierarchical features and complex representations by adding more fully connected layers.

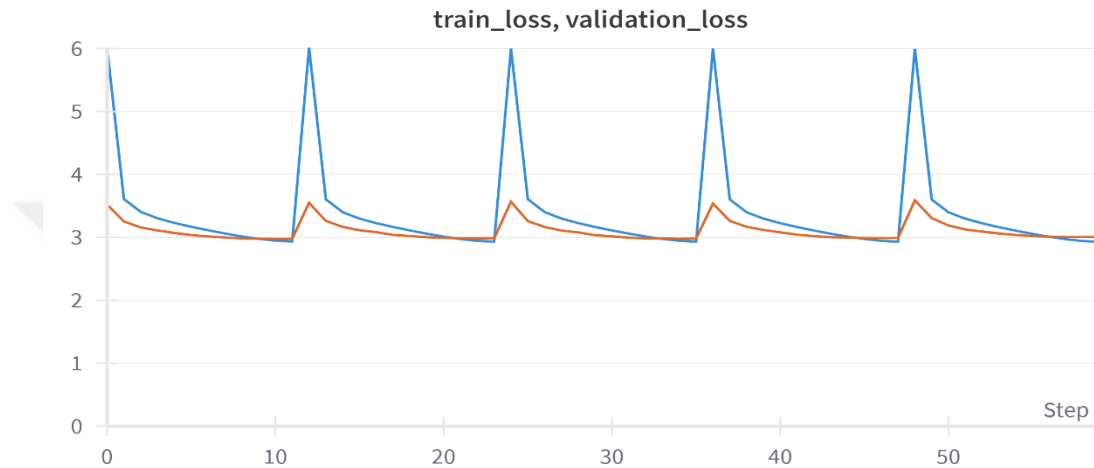


Figure 20: 1 FC Layer Added Model Train and Validation Loss Comparison

A new layer called Linear21 has been added between Linear2 and Linear3 layers. In Figure 20, layer added model's train and validation loss values can be observed. Train loss value 2.930 and validation loss value 3.009 have been obtained in this experiment.



Figure 21: 1 FC Layer Added Model Score

Figure 21 shows the scores of this run. It increases to 0.567328 with 27 minutes and 4 seconds running time.

5.2.4 2 Layers Added Model

When the score has increased by one layer added model, one more layer has been added in order to see if this new layer increases the score again. A new layer called Linear22 has been added between Linear21 and Linear 3.

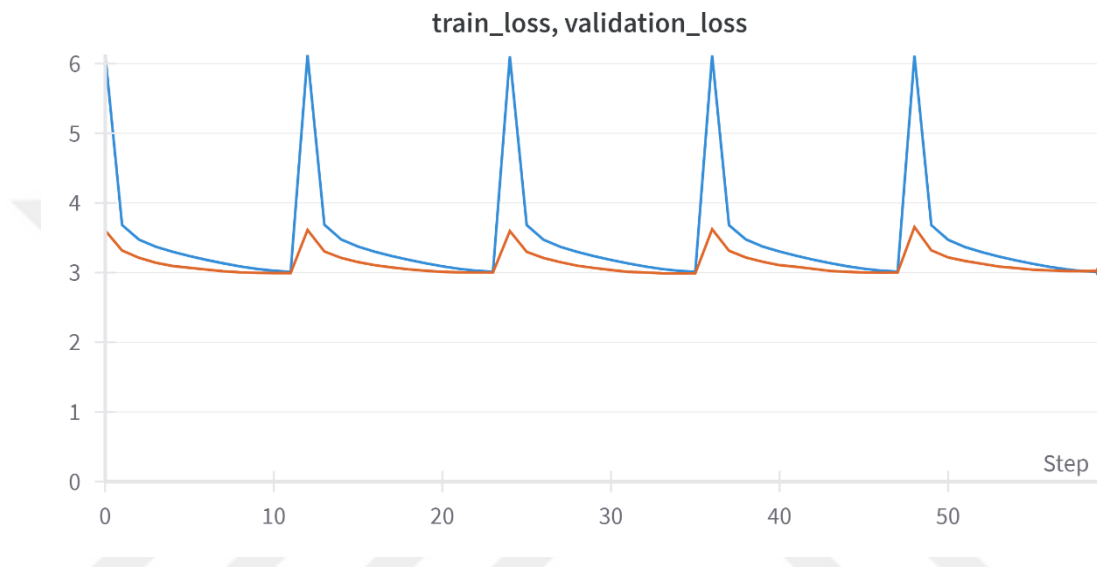


Figure 22: 2 FC Layers Added Model Train and Validation Loss Comparison

In Figure 22, 2 layer-added model is shown. Train loss has been computed at 2.966; validation loss has been computed at 3.009 in this experiment. Figure 23 shows the score values of two-layer added models. The score decreases to 0.565918 with 27 minutes 33 seconds run time in this experiment, as seen in Figure 23.

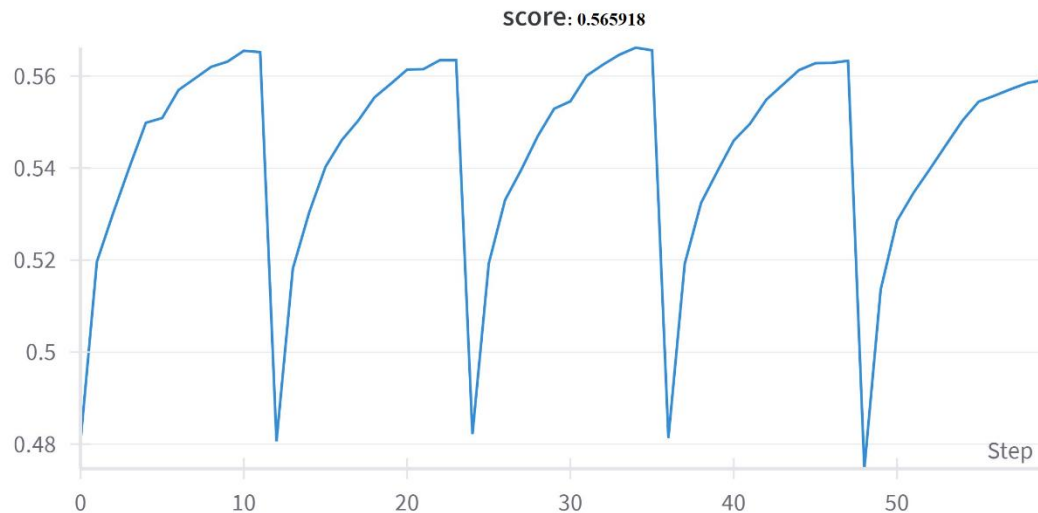


Figure 23: 2 FC Layers Added Model Score

5.2.5 RNN Added Model

Recurrent neural networks have been designed for sequential data. In our problem, the data used has sequential city trips. In a trip recommender model, where the order of cities is important, introducing a layer could enhance the model's ability to predict. Using recurrent neural networks by fully connected layers has been tried in other research and got some good results, as mentioned in previous chapters.

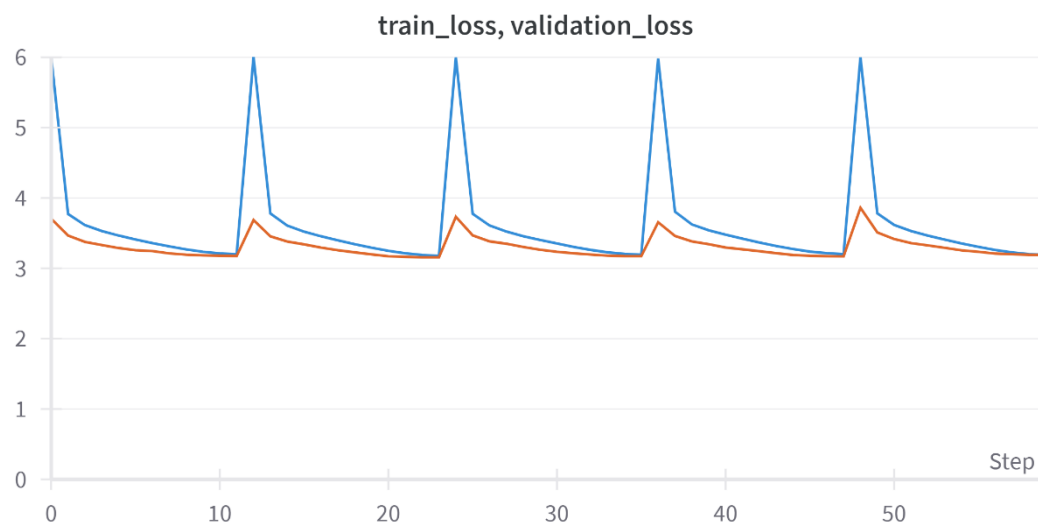


Figure 24: RNN Added Model Train and Validation Loss Comparison

The second fully connected layer has been removed, which the minimum effect on the learning has found by previous experiments, and a simple RNN layer is relocated instead of it. Figure 24 shows the values of train and validation losses. Train loss is 3.19, and validation loss is 3.193 in this experiment.

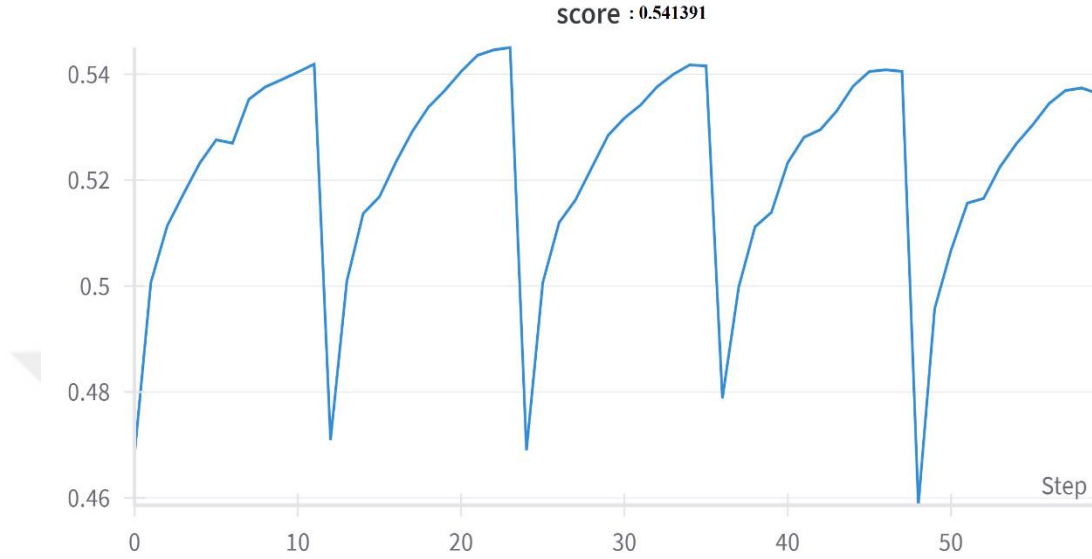


Figure 25: RNN Added Model Score

Figure 25 shows the scores of the simple RNN-added model. The score decreases to 0.541391 with a 1 hour 22 minutes 41 seconds run time in this experiment.

5.2.6 GRU Added Model

After getting a decreased score value by the simple RNN added model, a different recurrent model has been tested at this architecture. The simple RNN layer has been removed, and the GRU layer has been added instead to see improved recurrent model results.

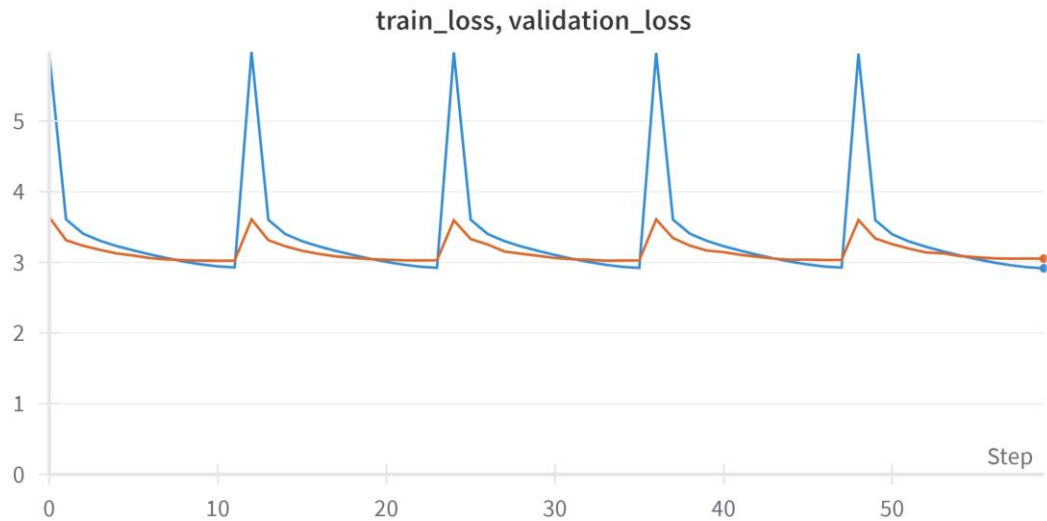


Figure 26: GRU Added Model Train and Validation Loss Comparison

In Figure 26, the GRU-added model's train and validation loss graphics can be observed. Train loss is 2.917, and validation loss is 3.054 in this experiment.

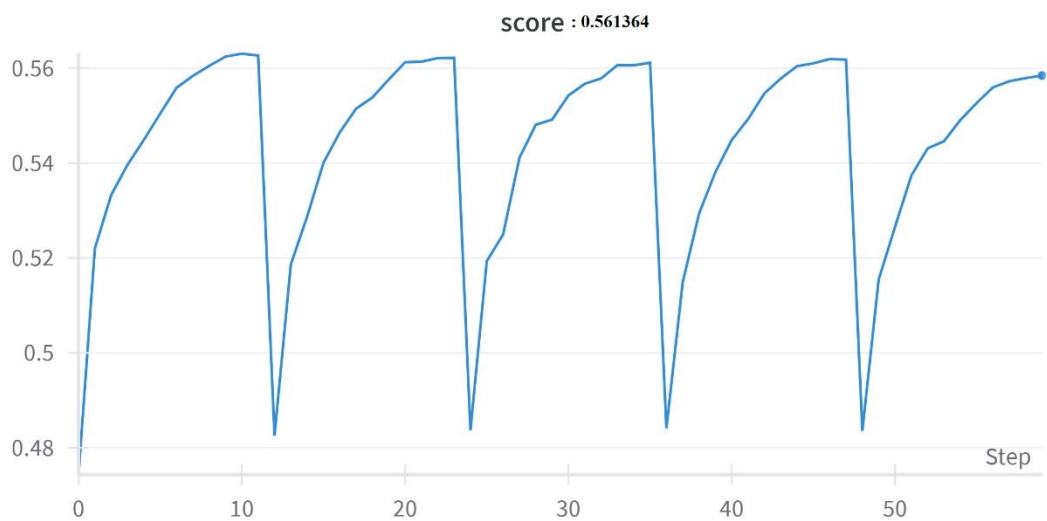


Figure 27: GRU Added Model Score

Figure 27 shows the result of the model's score. The score value is 0.561364 with 3 hours, 6 minutes, and 9 seconds running time.

5.2.7 Hyper Parameter Tuning Experiments

After finding the results of data and model experiments, hyperparameter tuning experiments have been made. Original data and 1 FC Layer Added model have been used in these experiments. Parameters and their values have been given in Figure 28. Some of these experiments have been shown before hyperparameter tuning; new experiments have been conducted to observe the effects of the combination of these parameters together. Hyperparameter tuning picks all parameter values that work best together and searches for high scores.

Over a hundred runs (experiments) have been made with hyperparameter tuning. In Figure 28, the results of best runs among hyperparameter tuning experiments can be seen. Using 12 epochs, 0.1688 as dropout, 0.00023358 as learning rate, 452 as train batch size, eight workers, Adadelata as optimizer, 1435 as hidden dimension, 67 as embedding dimension, 2 as gradient accumulation and BCEWithLogitsLoss as loss function has been given the best score which is 0.568032 with 2.8274 train loss value, 3.0409 validation loss value and 43 minutes 37 seconds run.

Score	Runtime	Dropout	Epochs	LR	train_loss	val_loss	train_batch_size	workers	optimizer	hidden_dim	embedding_dim	gradient_accumulation	loss_fct
0.568032	43m37s	0.1688	12	0.00023358	2.8274	3.0409	452	8	Adadelata	1435	67	2	BCEWithLogitsLoss
0.567976	43m7s	0.2009	13	0.00087881	2.8265	3.0424	163	14	SGD	1055	55	2	TripletMarginLoss
0.567945	41m31s	0.3996	23	0.00110539	2.8268	3.0426	442	15	Nadam	556	111	2	BCEWithLogitsLoss
0.567944	42m58s	0.4351	6	0.00076916	2.8265	3.0413	199	5	RMSProp	1473	97	1	BCEWithLogitsLoss
0.567904	44m8s	0.0729	23	0.00119349	2.8265	3.0369	129	13	AdaGrad	1892	76	1	TripletMarginLoss
0.567893	44m1s	0.1882	24	0.00081236	2.8270	3.0423	207	15	AdamW	901	104	1	TripletMarginLoss
0.567885	43m	0.8651	23	0.00068224	2.8267	3.0413	334	13	SGD	917	92	1	CrossEntropyLoss
0.567728	42m	0.8711	16	0.00159902	2.8267	3.0384	156	5	RMSProp	1837	128	1	BCEWithLogitsLoss
0.567907	43m42s	0.1377	15	0.00058241	2.8257	3.0408	129	4	Adam	1429	99	2	CrossEntropyLoss
0.567889	44m5s	0.2847	6	0.00027535	2.8274	3.0416	311	13	AdaGrad	1400	66	1	BCEWithLogitsLoss
0.567834	43m3s	0.9975	9	0.00179221	2.8266	3.0390	159	15	RMSProp	1654	123	2	HingeEmbeddingLoss

Figure 28: Best Scores in Hyper Parameter Tuning

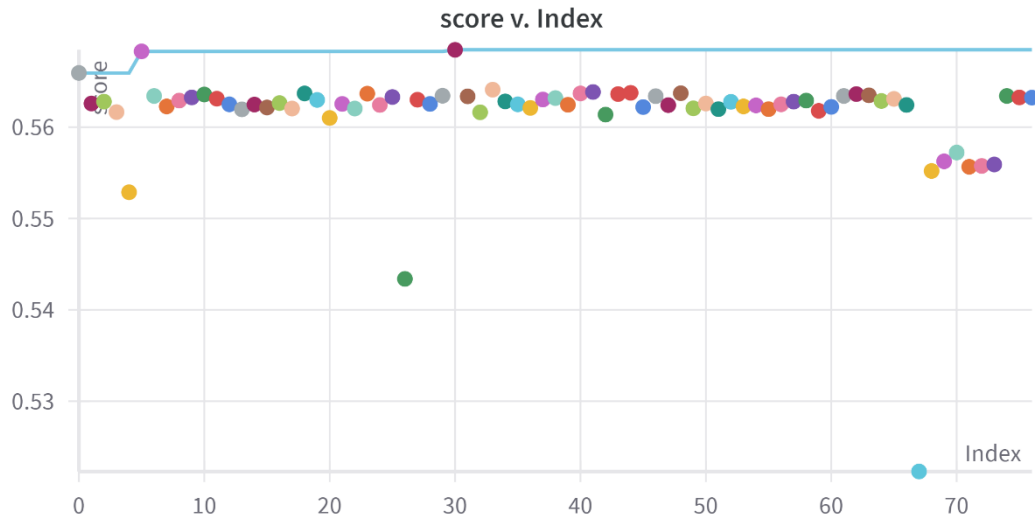


Figure 29: Overall Experiments in Hyper Parameter Tuning Score vs. Index of Runs

Figure 29 shows the overall experiments of hyperparameter tuning. The y-axis shows the score value, and the x-axis shows the index of runs, which means the order of experiments made. Each dot represents a run. The general distribution of runs is over 0.56, as seen. Some experiments' results are high, and some are low according to the overall distribution line.

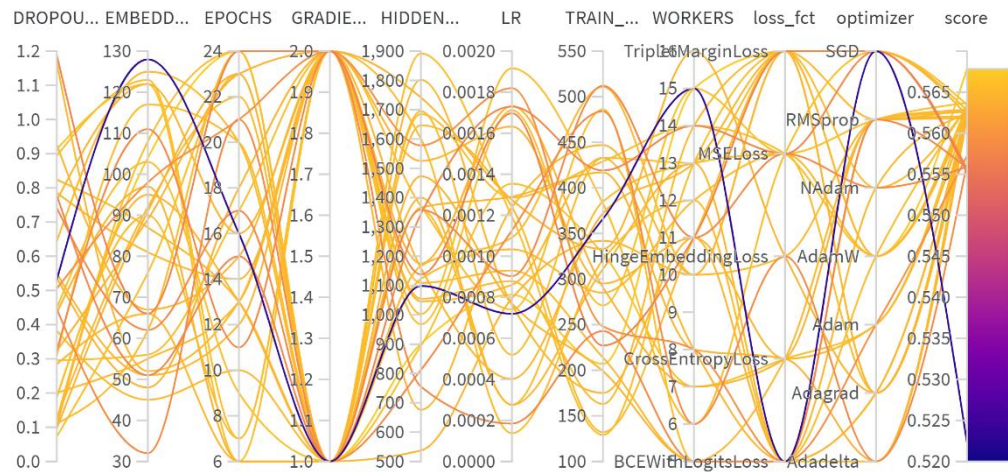


Figure 30: Overall Parameters and Values in Hyper Parameter Tuning

Figure 30 also shows the overall experiments of hyperparameter tuning. The X-axis shows the parameters used in the hyperparameter tuning yaml file; the y-axis

shows the values given to parameters. Each line represents a specific run. Lines visit the parameter values that they use.



CHAPTER VI

DISCUSSION AND CONCLUSION

This study can be divided into three sections. In the first section, data experiments have been discussed. In the second section, model and architecture experiments, and hyperparameter tuning experiments have been made in the third section. The results have been evaluated according to score, run time, training, and validation loss. If the score increases, it indicates that the prediction success of the model has increased. A high score, especially in applications such as recommendation systems, means one understands user preferences better and makes correct recommendations. If the score decreases, it means that the model's predictive ability has decreased, and it predicts user preferences less accurately. If the run time increases, training the model takes longer. This usually indicates that the model has become more complex. Increasing training time may not be practical, especially on large data sets, and increases resource usage. If it decreases, it shows that the model is trained faster. This usually happens by simplifying the model, which means less resource consumption. However, this does not always mean that the model is better. Sometimes, oversimplification can reduce the predictive ability of the model. If the training loss decreases, it indicates that the model performs better than the training data and captures the patterns in the data better. However, too low training loss may be a sign of overfitting, meaning the model may have become too specific to the training data and lost its ability to generalize. If the training loss increases, the model's performance on the training data has worsened. This may indicate that the model is experiencing underfitting or that the changes made have reduced the model's ability to understand the training data. If the validation loss decreases, the model performs better on data it has not seen, and its generalization ability increases. This usually means that the model will also perform well on real-world data. If it increases, it indicates that the model performs poorly on data other than training, and its generalization ability decreases. This may indicate an overlearning problem, meaning the model may have adapted very well to the training data but cannot adapt to new data.

The first section starts with data augmentation. The proposed study also expresses that data augmentation by reversing the trips and making them double has increased the model's ability to predict. Based on this idea, augmentation experiments have been made, too. Three iterations have been made to increase the number of trips. Three different CSV files have been obtained after these augmentations, and they have been used for different experiments with the original model, hoping to increase the score.

The score decreases to 0.554759 with 35 minutes and 48 seconds run time in the first iteration-dataset experiment. The second iteration-dataset score decreases to 0.548075 with 36 minutes and 11 seconds run time. In the third iteration, the dataset score decreases to 0.545146 with 36 minutes and 43 seconds run time.

As seen in the results of augmented datasets, the model's performance has decreased every time, contrary to our expectations. Running time increases in every experiment as expected. The added trips have a negative effect on the model's performance. After getting the results, it has been evaluated that the augmenting data could result in the unbalance of the datasets. Therefore, imbalanced data could fail to increase the accuracy. Table 6 shows the balance of datasets that have been used in the experiments. All datasets have been divided into five-folds, and each fold has a nearly equal number of rows. So, imbalanced data is not our problem, as seen in the table.

Nodes have been removed from longer trips and added as new shorter trips. Using this method in data augmentation negatively affects accuracy. This approach could cause the model to decrease the ability to learn.

Architecture experiments have been made in this study's second section/phase. The initial model with original data has been used as evaluation criteria and the results have been compared according to this. This model has three fully connected layers (FC Layers) in architecture. The second and third FC layers have been removed as different experiments to see the most ineffective layer. In the second FC layer removed model, the score has been decreased slightly with an acceptable runtime. In the third FC layer removed model the score has been decreased more dramatically with still an acceptable runtime.

According to the results, the second FC layer seems less effective in the architecture. The score has been decreased, and loss values have been increased in each removing experiment, but the second FC layer removed model is decided to be

used in the FC layer-GRU model and FC-layer simple RNN model because of a smaller decrease in score.

In the simple RNN added model, the score decreased again dramatically and is nearly the same as the score of the third FC layer removed model. However, the runtime has increased too much according to the initial and layer-removed models. In the GRU-added model, the score has been decreased according to the initial model. This model's score has been found to be higher than the simple RNN added model and third layer removed model, nearly the same as the second layer removed model. But the runtime has been increased too much according to all models. The runtime in this experiment has been found to increase three times according to the simple RNN model. Adding GRU instead of the second FC layer decreased the score and performance of the model. This model works six times slower than the original one. With the simple RNN modification, a better run time has been obtained according to GRU modification but a much-decreased score. Both models have reduced the performance and increased training time in our study.

These modifications in the model have not given the desired results. Adding FC layer/layers to the model's architecture has also been conducted as architecture experiments. One and two FC layers have been added to the model's architecture. In the one FC layer added experiment, the score has been increased slightly with nearly the same runtime. In the two FC layers added experiment, the score decreased slightly according to the original model, and one layer added a model with nearly the same runtime.

Adding one more FC layer to the architecture increased the score. This architecture has been found to be the most successful in the score among all experiments conducted on architecture and data. This indicates that the added layer slightly improved the performance of the model. Training time did not significantly increase. Adding one FC layer has increased the run time to only 13 seconds. This architecture also has a wellness in validation loss. This model has also decreased the validation loss, which means it works better with unfamiliar datasets. Adding two more FC layers to the architecture decreased the score and increased the complexity of the model.

It is seen that adding or removing layers affects the model's performance. Removing the third layer especially negatively affects the generalization of the model.

Adding more complex structures such as GRU and simple RNN to the model has significantly increased the training time. This is especially evident when GRU is used. Generally, GRU works well with time series or sequential data. In this case, using GRU has decreased the model's performance, which might suggest that using GRU may not be ideal between FC layers in our work. GRU is more computationally intensive. This long training time may be impractical, especially on large datasets or limited computational resources. Increasing training loss and validation loss indicate that GRU slightly reduces the model's generalization ability, meaning the model may be less successful on new data. Simple RNN is a simpler structure than GRU; in this case, it reduces the model's performance more. The results show that simple RNN is not powerful enough for this problem. Simple RNN runs faster than GRU, but in our model, this speed increase came with performance loss. Increased training and validation loss mean that the model performs worse on training and validation data, meaning the model's generalization ability has weakened.

Hyperparameter tuning experiments have been made after finding the optimal model and dataset. Wandb has been used, and a sweep for hyperparameter tuning was created. The most successful experiments have been selected and shown in experimental results. The best results have been gotten by using 12 epochs, 0.1688 as dropout, 0.00023358 as learning rate, 452 as train batch size, eight workers, Adadelta as an optimizer, 1435 as hidden dimension, 67 as embedding dimension, 2 as gradient accumulation and BCEWithLogitsLoss as loss function has been given the best score. Adding one FC layer and using these parameter values gave the best results in our work.

In conclusion, the data augmentation applied in the project has been conducted by focusing on trips with 4-10 city trips. The initial dataset consists of 1 545 502 rows and 287 533 trips. In each iteration, the data augmentation process creates new and unique trip sequences by removing one city from the beginning and the end of trips with six or more city trips and adding them to the original dataset. This method increases the size of the dataset in each iteration. According to the experimental results, the model's performance tends to decrease with increasing data. This decrease may be because more complex and diverse datasets make the learning process of the model more difficult. The model may have difficulty generalizing in increasing diversity or shortened trips. The experimental results show that data augmentation increases the amount of data and the diversity and complexity of the data. In travel recommendation

systems, the complexity of the relationships between different routes can affect the model's ability to learn and make accurate predictions. Therefore, the results obtained with data augmentation provide information about how the current configuration of the model reacts to more complex data sets and the importance of longer sequences. These experiments play an important role in evaluating the effect of data augmentation on model performance. The diversity and complexity have been introduced by data augmentation to test the model's ability to generalize and provide opportunities for potential improvements. However, we should be careful in this process, and additional adjustments may need to be made to ensure the model can overcome complex datasets. This part of the study shows how data augmentation increases the amount of data and how it can also change the learning dynamics of the model.

The results of experiments conducted on a deep learning-based recommendation system with various configurations have been examined in detail in this study. The initial model has a three-layer multilayer perceptron structure, modified throughout the experiments, and the system's performance has been measured. First, the effects of removing the middle and third layers, respectively, have been evaluated. Removing the second layer has led to a slight decrease in the model's score and an increase in running time while removing the third layer resulted in a more significant decrease in the score and an even greater increase in running time. This shows that decreasing the number of layers negatively affects the overall performance of this model. Next, adding GRU and simple RNN layers instead of the second layer has been tried. The addition of GRU has significantly increased the runtime while keeping the model's score close but lower according to the original. Adding simple RNN also reduces the score and increases the running time. These experiments show that GRU and simple RNN are ineffective in this particular structure between FC layers. Finally, the model's performance has been tested by adding extra linear layers to the original model. The first extra layer (Linear 21) has slightly increased the model's score while leaving the running time almost unchanged. Adding two extra layers (Linear 21 and Linear 22) has resulted in a similar runtime and a slightly decreased score. In the overall evaluation of these experiments, the model created with the addition of Linear 21 has presented the best results with balanced performance and acceptable runtime. This model stands out as the optimal structure, offering a slightly improved score while maintaining the stability provided by the original model. Other configurations either have caused a decrease in performance or significantly increased runtime. This part of

the study reveals the impact of model configurations on the effectiveness of recommendation systems and emphasizes the importance of a balanced approach in model selection.

As a summary of the results of the architecture experiments, the initial structure of the model seems balanced and efficient. Removing layers negatively affects overall performance, but adding an extra layer provides a slight improvement. Trying different frameworks, such as GRU and simple RNN, may work in some cases but has not provided a significant improvement in this particular case. These results indicate a slight improvement in one layer of added architecture, but tuning the hyperparameters of our new architecture (e.g., learning rate and batch size) can also improve performance.

The results obtained in the hyperparameter tuning experiments show several factors that influence the machine learning model's performance. First, the scores obtained are in a close range (0.567834-0.568032), indicating that the experiments provide generally consistent results. However, the fact that the highest score has been achieved in a relatively long but still acceptable running time of 43 minutes and 37 seconds shows that a careful balance should be established between time efficiency and performance. The importance of the dropout rate is also high. The best-performing model has used a medium dropout rate (0.1688). This shows us that the dropout rate shouldn't be chosen too high or too low. Other factors that affect the learning process, such as the number of epochs and learning rate (LR), are also important. The best results have been obtained with epoch numbers and LR values that are not too high or too low. It requires a balanced approach to prevent overfitting while ensuring the model has adequate learning opportunities. The closeness of the train and validation loss values to each other shows that the model performs good in both the training and validation sets. In comparing different optimizers, Adadelata has produced the best results for this set of experiments, showing that some optimizers may be more effective than others for a given dataset and model architecture. Optimization of hidden dimension, embedding dimension, and other parameters is also essential. These experiments show the true combination of parameter values can significantly impact the model's overall performance.

REFERENCES

- [1] MCDONALD Carol and G. MOREIRA Gabriel (2021), *How to Build a Winning Recommendation System Part 1*, <https://developer.nvidia.com/blog/how-to-build-a-winning-recommendation-system-part-1/>, DoA. 15.11.2023.
- [2] BOBADILLA Jesus, ORTEGA Fernando, HERNANDO Antonio, and GUTIERREZ Abraham (2013), “Recommender systems survey”, *Knowl. Based Syst.*, Vol. 46, pp. 109–132
- [3] BURKE Robin, FELFERNIG Alexander, and GÖKER Mehmet H. (2011), “Recommender systems: An overview”, *AI Mag.*, Vol. 32, No. 3, pp. 13–18.
- [4] CHEN Rui, HUA Qingyi, CHANG Yan-Shuo, WANG Bo, L. Zhang, and X. KONG Xiangjie (2018), “A survey of collaborative filtering-based recommender systems: From traditional methods to hybrid methods based on social networks”, *IEEE Access*, Vol. 6, pp. 64301–64320.
- [5] SCHAFER Ben, FRANKOWSKI Dan, HERLOCKER John, and SEN Shilad (2007), *The adaptive web: methods and strategies of web personalization*, Springer, Berlin Heidelberg.
- [6] METEREN Van Robin and SOMEREN Van Maarten (2000), “Using Content-Based Filtering for Recommendation”, *Workshop, in Proceedings of the Machine Learning in the New Information Age: MLnet/ECML2000*, Barcelona.
- [7] BASILICO Justin and HOFMANN Thomas (2004), “Unifying collaborative and content-based filtering”, *In, Proceedings of the Twenty-First International Conference on Machine Learning*, pp. 9, Association for Computing Machinery, New York.
- [8] Zhang Shuai, YAO Lina, SUN Aixin, and TAY Yi (2019), “Deep learning based recommender system: A survey and new perspectives”, *ACM Comput. Surv.*, Vol. 52, No. 1, pp. 1–38.
- [9] CHENG Heng-Tze, KOC Levent, HARMSSEN Jeremiah, SHAKED Tal, CHANDRA Tushar, ARADHYE Hrishi, ANDERSON Glen, CORRADO Greg, CHAI Wei, ISPIR Mustafa, ANIL Rohan, HAQUE Zakaria, HONG Lichan, JAIN Vihan, LIU Xiaobing, and SHAH Hemal (2016), “Wide & deep learning for recommender systems”, *In, Proceedings of the 1st Workshop on Deep Learning for Recommender Systems*, pp. 7-10, Association for Computing Machinery, New York.

- [10] STECK Herald, BALTRUNAS Linas, ELAHI Ehtsham, LIANG Dawen, RAIMOND Yves, and BASILICO Justin (2021), “Deep learning for recommender systems: A Netflix case study”, *AI Mag.*, Vol. 42, No. 3, pp. 7–18.
- [11] MU Ruihui (2018), “A survey of recommender systems based on deep learning”, *IEEE Access*, Vol. 6, pp. 69009–69022.
- [12] COVINGTON Paul, ADAMS Jay, and SARGIN Emre (2016), “Deep neural networks for YouTube recommendations”, In, *Proceedings of the 10th ACM Conference on Recommender Systems*, pp. 191-198, Association for Computing Machinery, New York.
- [13] BOBADILLA Jesus, ALONSO Santiago, and HERNANDO Antonio (2020), “Deep learning architecture for collaborative filtering recommender systems”, *Appl. Sci.*, Vol. 10, No. 7, p. 2441.
- [14] TOMAR Abhineswar, GODIN Frederic, VANDERSMISSEN Baptist, DE NEVE Wesley, and VAN DE WALLE Rik (2014), “Towards Twitter hashtag recommendation using distributed word representations and a deep feed-forward neural network”, *2014 International Conference on Advances in Computing, Communications, and Informatics (ICACCI)*, pp. 362-368, India.
- [15] ZUO Yi, ZENG Jiulin, GONG Maoguo, and JIAO Licheng (2016), “Tag-aware recommender systems based on deep neural networks”, *Neurocomputing*, Vol. 204, pp. 51–60.
- [16] XU Zhenghua, CHEN Cheng, LUKASIEWICZ Thomas, MIAO Yishu, and MENG Xiangwu (2016), “Tag-aware personalized recommendation using a deep-semantic similarity model with negative sampling”, In, *Proceedings of the 25th ACM International on Conference on Information and Knowledge Management*, pp. 1921-1924, Association for Computing Machinery, New York.
- [17] GUPTA Garima and KATARYA Rahul (2021), “Research on understanding the effect of deep learning on user preferences”, *Arab. J. Sci. Eng.*, Vol. 46, No. 4, pp. 3247–3286.
- [18] GANAIE Mudashir Ahmad, HU Minghui, MALIK Ashwani Kumar, TANVEER Mohammad, and SUGANTHAN Ponnuthurai (2022), “Ensemble deep learning: A review”, *Eng. Appl. Artif. Intell.*, Vol. 115, No. 105151, pp. 10515.

- [19] CARUANA Rich and MIZIL Alexandru Niculescu (2006), “An empirical comparison of supervised learning algorithms”, *In, Proceedings of the 23rd International Conference on Machine Learning - ICML '06*, pp.161-168, Association for Computing Machinery, New York.
- [20] DIETTERICH Thomas (2000), “Ensemble Methods in Machine Learning”, *in Multiple Classifier Systems*, Vol. 1857, pp. 1–15.
- [21] HASTIE Trevor, TIBSHIRANI Robert, and FRIEDMAN Jerome (2009), *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, Springer, New York.
- [22] R. Polikar (2009), *Ensemble learning*, http://www.scholarpedia.org/article/Ensemble_learning, DoA. 27.11.2023.
- [23] HUANG Yin-Fu and CHEN Po-Hong (2020), “Fake news detection using an ensemble learning model based on Self-Adaptive Harmony Search algorithms”, *Expert Syst. Appl.*, Vol. 159, p. 113584.
- [24] SOLOVYEV Roman, WANG Weimin, and T. GABRUSEVA Tatiana (2021), “Weighted boxes fusion: Ensembling boxes from different object detection models”, *Image Vis. Comput.*, Vol. 107, p. 104117.
- [25] FOROUZANDEH Saman, BERAHMAND Kamal, and ROSTAMI Mehrdad (2021), “Presentation of a recommender system with ensemble learning and graph embedding: a case on MovieLens”, *Multimed. Tools Appl.*, Vol. 80, No. 5, pp. 7805–7832.
- [26] ZHOU Yunhong, WILKINSON Dennis, SCHREIBER Robert, and PAN Rong (2008), “Large-scale parallel collaborative filtering for the Netflix prize”, *In, Algorithmic Aspects in Information and Management*, Ed. Rudolf Fleischer, Jinhui Xu, pp. 337–348, Springer Berlin Heidelberg, Berlin.
- [27] LIU Hongzhi, DU Yingpeng, and WU Zhonghai (2022), “Generalized Ambiguity Decomposition for Ranking Ensemble Learning”, *J. Mach. Learn. Res.*, Vol. 23, No. 88, pp. 1-36.
- [28] ZHU Jun, HAN Lixin, GOU Zhinan, YANG Yi, YUAN Xiaofeng, LI Jingxian, and LI Shu (2021), “A robust personalized location recommendation based on ensemble learning”, *Expert Syst. Appl.*, Vol. 167, pp. 114065.
- [29] FAN Cheng, SUN Yongjun, ZHAO Yang, SONG Mengjie, and WANG Jiayuan (2019), “Deep learning-based feature engineering methods for improved building energy prediction”, *Appl. Energy*, vol. 240, pp. 35–45.

- [30] LONG Wen, LU Zhichen, and CUI Lingxiao (2019), “Deep learning-based feature engineering for stock price movement prediction”, *Knowl. Based Syst.*, Vol. 164, pp. 163–173.
- [31] MISHRA Puneet, BIANCOLILLO Alessandra, ROGER Jean Michel, MARINI Federico, and RUTLEDGE Douglas (2020), “New data preprocessing trends based on ensemble of multiple preprocessing techniques”, *Trends Analyt. Chem.*, Vol. 132, No. 116045, pp. 116045.
- [32] ALEXANDROPOULOS Stamatios Aggelos, KOTSIANTIS Sotiris, and VRAHITIS Michael (2019), “Data preprocessing in predictive data mining”, *Knowl. Eng. Rev.*, Vol. 34, pp. e1.
- [33] DASH Manoranjan and LIU Huan (1997), “Feature selection for classification”, *Intell. Data Anal.*, Vol. 1, issue 1–4, pp. 131–156.
- [34] KHAIRE Utkarsh Mahadeo and DHANALAKSHMI (2022), “Stability of feature selection algorithm: A review”, *J. King Saud Univ. - Comput. Inf. Sci.*, Vol. 34, No. 4, pp. 1060–1073.
- [35] CHANDRASHEKAR Girish and SAHIN Ferat (2014), “A survey on feature selection methods”, *Comput. Electr. Eng.*, Vol. 40, No. 1, pp. 16–28.
- [36] BENNASAR Mohamed, HICKS Yulia, and SETCHI Rossitza (2015), “Feature selection using Joint Mutual Information Maximisation”, *Expert Syst. Appl.*, Vol. 42, No. 22, pp. 8520–8532.
- [37] GUO Lei, YIN Hongzhi, WANG Qinyong, CHEN Tong, ZHOU Alexander, and HUNG Nguyen Quoc Viet (2019), “Streaming session-based recommendation”, In, *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pp. 1569-1577, Association for Computing Machinery, New York.
- [38] WANG Shoujin, CAO Longbing, WANG Yan, SHENG Quan, ORGUN Mehmet, and LIAN Defu (2022), “A survey on session-based recommender systems”, *ACM Comput. Surv.*, Vol. 54, No. 7, pp. 1–38.
- [39] SAKATANI Yoshihiro (2021), “Session-Based Recommendation Using an Ensemble of LSTM- and Matrix Factorization-Based Models”, *Proceedings of ACM SIGIR Workshop on eCommerce*, Montreal, Canada.

- [40] DESAI Meha and SHAH Manan (2021), “An anatomization on breast cancer detection and diagnosis employing multi-layer perceptron neural network (MLP) and Convolutional neural network (CNN)”, *Clin. eHealth*, Vol. 4, pp. 1–11.
- [41] ALMEIDA Luis (1996), *Handbook of Neural Computation*, First Edition, CRC Press.
- [42] FINZI Marc, WELLING Max, and WILSON Andrew Gordon (2021), “A practical method for constructing equivariant multilayer perceptrons for arbitrary matrix groups”, In, *Proceedings of the 38th International Conference on Machine Learning*, Ed. Marina Meila, Tong Zhang, pp. 3318–3328, PMLR, California.
- [43] KIRAN, KUMAR Pradeep, and BHASKER Bharat (2020), “DNNRec: A novel deep learning-based hybrid recommender system”, *Expert Syst. Appl.*, Vol. 144, p. 113054.
- [44] KHALEDIAN Navid, NAZARI Amin, KHAMFOROOSH Keyhan, ABUALIGAH Laith, and JAVAHERI Daniel (2023), “TrustDL: Use of trust-based dictionary learning to facilitate recommendation in social networks”, *Expert Syst. Appl.*, Vol. 228, p. 120487.
- [45] SCHEDL Markus (2019), “Deep learning in music recommendation systems”, *Front. Appl. Math. Stat.*, Vol. 5, Article 44, pp. 1-9.
- [46] CHOUDHARY Chaitali, SINGH Inder, and KUMAR Manoj (2023), “SARWAS: Deep ensemble learning techniques for sentiment-based recommendation system”, *Expert Syst. Appl.*, Vol. 216, p. 119420.
- [47] LIU Tiejuan, WU Qiong, CHANG Liang, and GU Tianlong (2022), “A review of deep learning-based recommender system in e-learning environments”, *Artif. Intell. Rev.*, Vol. 55, No. 8, pp. 5953–5980.
- [48] LEE Hanbit, AHN Yeonchan, LEE Haejun, HA Seungdo, and LEE Sang-goo (2016), “Quote recommendation in dialogue using deep neural network”, In, *Proceedings of the 39th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 957-960, Association for Computing Machinery, New York.

- [49] DING Daizong, ZHANG Mi, LI Shao-Yuan, TANG Jie, CHEN Xiaotie, and ZHOU Zhi-Hua (2017), “BayDNN: Friend recommendation with Bayesian personalized ranking deep neural network”, *In, Proceedings of the 2017 ACM on Conference on Information and Knowledge Management*, pp. 1479-1488, Association for Computing Machinery, New York.
- [50] BAI Bing, FAN Yushun, TAN Wei, and ZHANG Jia (2020), “DLTSR: A deep learning framework for recommendations of long-tail web services”, *IEEE Trans. Serv. Comput.*, Vol. 13, No. 1, pp. 73–85.
- [51] ZHANG Libo, LUO Tiejian, ZHANG Fei, and WU Yanjun (2018), “A recommendation model based on deep neural network”, *IEEE Access*, Vol. 6, pp. 9454–9463.
- [52] PARK Keunchan, LEE Jisoo, and CHOI Jaeho (2017), “Deep neural networks for news recommendations”, *In, Proceedings of the 2017 ACM Conference on Information and Knowledge Management*, pp. 2255-2258, Association for Computing Machinery, New York.
- [53] WU Sai, REN Weichao, YU Chengchao, CHEN Gang, D. ZHANG Dongxiang, and ZHU Jingbo (2016), “Personal recommendation using deep recurrent neural networks in NetEase”, *In, 2016 IEEE 32nd International Conference on Data Engineering (ICDE)*, pp.1218-1229, IEEE, Helsinki.
- [54] HUANG Zhenhua, YU Chang, NI Juan, LIU Hai, ZENG Chun, and TANG Yong (2019), “An efficient hybrid recommendation model with deep neural networks”, *IEEE Access*, Vol. 7, pp. 137900–137912.
- [55] MOREIRA Gabriel de Souza Pereira, FERREIRA Felipe, and CUNHA Adilson Marques (2018), “News session-based recommendations using deep neural networks”, *In, Proceedings of the 3rd Workshop on Deep Learning for Recommender Systems*, pp. 15-23, Association for Computing Machinery, New York.
- [56] ALASHKAR Taleb, JIANG Songyao, WANG Shuyang, and FU Yun (2017), “Examples-rules guided deep neural network for makeup recommendation”, *Proc. Conf. AAAI Artif. Intell.*, Vol. 31, pp. 941-947, No. 1.
- [57] ZHANG Wen, DU Yuhang, TAKETOSHI Yoshida, and YANG Ye (2019), “DeepRec: A deep neural network approach to recommendation with item embedding and weighted loss function”, *Inf. Sci. (NY)*, Vol. 470, pp. 121–140.

- [58] ZHU Qiannan, ZHOU Xiaofei, SONG Zeliang, TAN Jianlong, and GUO Li (2019), “DAN: Deep attention neural network for news recommendation”, *Proc. Conf. AAAI Artif. Intell.*, Vol. 33, No. 01, pp. 5973–5980.
- [59] NASSAR Nour, JAFAR Assef, and RAHHAL Yasser (2020), “A novel deep multi-criteria collaborative filtering model for recommendation system”, *Knowl. Based Syst.*, Vol. 187, p. 104811.
- [60] GOLDENBERG Dmitri and LEVIN Pavel (2021), “Booking.com Multi-Destination Trips Dataset”, In, *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 2457-2462, Association for Computing Machinery, New York.
- [61] HU Hengyuan, PENG Rui, TAI Yu-Wing, and TANG Chi-Keung (2016), “Network trimming: A data-driven neuron pruning approach towards efficient deep architectures”, *CoRR*, Vol. abs/1607.03250, DOI:10.48550/arXiv.1607.03250.
- [62] UZAIR Muhammed and JAMIL Noreen (2020), “Effects of Hidden Layers on the Efficiency of Neural Networks”, In, *2020 IEEE 23rd International Multitopic Conference (INMIC)*, pp. 1-6, IEEE, Lahore.
- [63] ZHENG Tianyou, WANG Qiang, SHEN Yue, and LIN Xiaotian (2022), “Gradient rectified parameter unit of the fully connected layer in convolutional neural networks”, *Knowl. Based Syst.*, Vol. 248, p. 108797.